Micro-ROS Sensor Node

on Renesas EK-RA6M5

Complete Build, Debug, and Deployment Guide

Table of Contents

Table of Contents 2
1. The Problem Micro-ROS Solves 6
1.1 What Is ROS 2? 6
1.2 The Microcontroller Problem 6
1.3 What Micro-ROS Does 7
2. How Micro-ROS Works 8
2.1 The Architecture 8
2.2 The Transport Layer 8
2.3 The Client Library Stack 9
2.4 The Agent 10
3. What Micro-ROS Does in This Project 11
3.1 The Without-Micro-ROS Baseline 11
3.2 What Micro-ROS Changes 11
3.3 The Data Flow 12
3.4 Micro-ROS vs. Alternatives 13
3.5 What Micro-ROS Requires from the Firmware 13
4. Project Overview 16
4.1 Hardware Bill of Materials 16
4.2 Software Stack 16
4.3 I2C Wiring 17
5. Phase A: Bare-Metal USB Version 18
5.1 Create the e² Studio Project 18
5.2 Configure FSP Peripherals 18
5.2.1 I2C Master 18
5.2.2 USB PCDC 18
5.2.3 Pins 19
5.3 Application Code: hal_entry.c 19
5.4 Build, Flash, and Verify 22
6. Phase B: Micro-ROS Version 24
6.1 Create the Project 24
6.2 FSP Configuration 24
6.2.1 I2C Master 24
6.2.2 USB PCDC 24

6.2.3 FreeRTOS Thread 24
6.2.4 Critical: FreeRTOS Heap Size 24
6.3 Building libmicroros.a 25
6.3.1 What Did NOT Work 25
6.3.2 What Worked 25
6.3.3 Installing the Library 26
6.4 Source Files 26
7. Complete Source Code Listings (Micro-ROS Version) 28
7.1 new_thread0_entry.c 28
7.2 sensors.h 34
7.3 sensors.c 34
7.4 microros_transports.h 38
7.5 microros_transports.c 38
7.6 microros_stubs.c 42
7.7 freertos_malloc_override.c 44
7.8 usb_descriptor.c 47
8. Debugging: What Went Wrong and How It Was Fixed 50
8.1 Bug #1: Broken malloc (rc=10) 50
8.2 Bug #2: FreeRTOS Heap = 1024 Bytes 50
8.3 Bug #3: rclc_support_init Returns rc=1 50
8.4 Bug #4: USB Device Disappearing 51
8.5 Bug #5: Permission Denied (errno 13) 51
9. Host-Side Setup: WSL2, ROS 2, and the Agent 52
9.1 Install WSL2 with Ubuntu 22.04 52
9.2 Install ROS 2 Humble 52
9.3 Build the Micro-ROS Agent 53
9.4 USB Passthrough (usbipd-win) 53
9.5 Run the Agent and Verify 54
10. Deployment: Podman Container 55
10.1 Containerfile 55
10.2 Build and Run 55
11. Troubleshooting Quick Reference 56
12. Lessons Learned 58
12.1 USB Polling Is Non-Negotiable 58
12.2 Fix the Allocator at Every Level 58

12.3 Compiler -D Flags Can Be Silently Ignored 58
12.4 WSL2 Is Not Linux 58
12.5 Budget a Full Day for Library Generation 59
13. Project File Reference 60
13.1 Bare-Metal Project 60
13.2 Micro-ROS Project 60
14. Glossary 61

Part I: What Is Micro-ROS?

1. The Problem Micro-ROS Solves

1.1 What Is ROS 2?

ROS 2 (Robot Operating System 2) is an open-source framework used throughout robotics, autonomous vehicles, industrial automation, and research. It is not an operating system in the traditional sense. It is a communication framework: a standardized way for software components (called "nodes") to discover each other, exchange data, and coordinate behavior across a network.

Where to find it: ROS 2 Humble documentation: https://docs.ros.org/en/humble/ — Installation guide (Ubuntu): https://docs.ros.org/en/humble/Installation/Ubuntu-Install-Debs.html — Standard message types: https://github.com/ros2/common_interfaces

A ROS 2 system is composed of nodes that publish and subscribe to named data channels called "topics." A temperature sensor node might publish to a topic called /sensors/room_temp. A climate control node might subscribe to that same topic and use the data to decide whether to turn on a heater. Neither node needs to know where the other is running — they could be on the same computer, on different computers, or on different continents. ROS 2 handles the discovery and transport automatically through a middleware layer called DDS (Data Distribution Service).

For example, in a robotic arm system you might have separate nodes for joint motor control, force sensing, path planning, and a user interface. Each node runs independently, and they communicate by publishing messages to topics or calling services on other nodes. This decoupled architecture means you can restart one node without affecting the others, swap implementations, or add new nodes to the system without modifying existing ones.
ROS 2 also provides quality-of-service (QoS) settings on each topic, letting you configure reliability (best-effort vs. reliable delivery), durability (transient-local vs. volatile), and history depth. These settings matter for real-time sensor data — you typically want best-effort delivery so a slow subscriber does not cause the publisher to block.

ROS 2 runs on full-featured operating systems like Ubuntu Linux. It requires a POSIX-compatible environment, dynamic memory allocation, multithreading, a network stack, and typically several hundred megabytes of RAM. A standard ROS 2 installation consumes gigabytes of disk space.

1.2 The Microcontroller Problem

Microcontrollers are the opposite of that environment. A typical microcontroller like the Renesas RA6M5 used in this project has 512 KB of SRAM and 2 MB of flash storage. It runs bare-metal code or a lightweight RTOS like FreeRTOS. It has no operating system in the traditional sense, no filesystem, no network stack beyond what you build yourself, and no concept of processes or dynamic linking.

This creates a fundamental gap. The sensors, actuators, and embedded controllers that robots and industrial systems depend on run on microcontrollers. The high-level decision-making, data logging, visualization, and coordination run on full computers with ROS 2. Getting data from one world to the other traditionally required writing custom serial protocols, custom parsers, custom message formats, and custom bridging code. Every project reinvented this wheel.

To put this in concrete terms: a standard ROS 2 node on Ubuntu typically uses 20-50 MB of RAM just for the DDS middleware alone. The RA6M5 has 0.5 MB total. A full ROS 2 installation requires hundreds of

shared libraries, a filesystem, thread scheduling from a POSIX OS, and dynamic linking — none of which exist on a bare-metal microcontroller. You cannot simply cross-compile ROS 2 for a Cortex-M chip; the architecture is fundamentally incompatible.
Yet microcontrollers are where the physical world meets the software. They read sensors, drive actuators, and respond to real-time events with microsecond-level determinism that Linux cannot guarantee. Any robotics system that interacts with the physical world needs both: Linux for planning and coordination, microcontrollers for sensing and actuation.

1.3 What Micro-ROS Does

Micro-ROS bridges this gap. It is a stripped-down implementation of the ROS 2 client libraries, specifically designed to run on microcontrollers with as little as 32 KB of RAM. It allows a microcontroller to act as a first-class ROS 2 node: it can publish topics, subscribe to topics, offer services, and participate in the ROS 2 ecosystem exactly as if it were a full Linux machine running standard ROS 2.

The key insight is that micro-ROS does not try to run the full DDS middleware on the microcontroller. Instead, it uses a lightweight serialization protocol called XRCE-DDS (eXtremely Resource Constrained Environments DDS). The microcontroller communicates through a thin serial link to a "micro-ROS agent" running on a host computer. The agent translates between XRCE-DDS and full DDS, making the microcontroller’s topics appear on the ROS 2 network as if they originated from a native node.

XRCE-DDS stands for "eXtremely Resource Constrained Environment DDS." It was designed by eProsima specifically for microcontrollers. Instead of implementing the full DDS standard (which requires UDP multicast, participant discovery, and complex QoS negotiation), XRCE-DDS uses a client-server model. The microcontroller runs a thin client that sends compact binary frames to an agent process on a Linux host. The agent translates these frames into full DDS messages, making the microcontroller appear as a native ROS 2 participant on the network.
The practical result: your microcontroller firmware calls functions like rcl_publish() that look almost identical to what you would write in a Linux ROS 2 node. The complexity of DDS, serialization, and network transport is hidden behind the same API. The tradeoff is that the microcontroller cannot operate independently — it always needs the agent running on a host to bridge into the ROS 2 network.

Key Concept
From the perspective of any other ROS 2 node on the network, there is no difference between a topic published by a desktop computer running Ubuntu and a topic published by a microcontroller running micro-ROS. The data types are the same, the discovery mechanism is the same, the quality-of-service settings are the same. The microcontroller is a full participant in the ROS 2 graph.

2. How Micro-ROS Works

2.1 The Architecture

A micro-ROS system has three layers:

Layer 1 — The Microcontroller: runs the micro-ROS client library (compiled into libmicroros.a) inside a FreeRTOS task. This layer handles sensor reading, message creation, serialization, and sending frames over the transport (USB in this project). It has no knowledge of the full ROS 2 network.
Layer 2 — The Agent: a Linux process that maintains a bidirectional connection to the microcontroller over the transport. It translates XRCE-DDS frames into full DDS messages and vice versa. The agent creates DDS participants, topics, and datawriters on behalf of the microcontroller. From the perspective of other ROS 2 nodes on the network, the microcontroller's topics look like they come from a native Linux publisher.
Layer 3 — The ROS 2 Network: standard ROS 2 nodes running on one or more Linux machines, communicating via full DDS. These nodes have no idea they are receiving data from a microcontroller — the agent's translation is transparent.

Layer	Runs On	What It Does
Micro-ROS Client	Microcontroller (RA6M5)	Creates nodes, publishers, subscribers, timers. Serializes messages into XRCE-DDS frames. Sends/receives frames over a transport (USB, UART, Wi-Fi, etc.)
Micro-ROS Agent	Host computer (Linux)	Receives XRCE-DDS frames from the serial link. Translates them into full DDS messages. Acts as a proxy for the microcontroller on the ROS 2 network.
ROS 2 Network	Any machine on the network	Standard ROS 2 nodes that subscribe to, publish to, or interact with the microcontroller’s topics. Completely unaware that the data originates from a microcontroller.

The communication between the microcontroller and the agent is bidirectional. The agent does not simply relay data — it manages the lifecycle of the microcontroller’s ROS 2 entities. When the micro-ROS client creates a publisher, the agent creates a corresponding real DDS publisher on the host side. When the client publishes a message, the agent receives the serialized bytes, deserializes them, and republishes through the real DDS publisher. The reverse happens for subscriptions.

2.2 The Transport Layer

Micro-ROS is transport-agnostic. The client library defines four callback functions: open, close, write, and read. Any physical link that can move bytes can serve as a transport. Common options include UART, USB-CDC, Wi-Fi, Ethernet, and CAN bus.

In this project, the transport is USB-CDC (Communications Device Class). The microcontroller presents itself to the host as a virtual serial port. The agent opens this serial port and exchanges XRCE-DDS frames

over it. USB-CDC was chosen because the EK-RA6M5 has a USB Full Speed peripheral, providing higher throughput and lower latency than UART.

USB-CDC (Communications Device Class) makes the microcontroller appear as a virtual serial port on the host computer. On Linux, it shows up as /dev/ttyACM0 (or ttyACM1, ttyACM2, etc. if multiple CDC devices are connected). No special driver is needed — the Linux kernel's built-in cdc_acm driver handles it automatically.
The four transport callbacks in this project are implemented in microros_transports.c (Section 7.5): cdc_transport_open() initializes the USB stack and waits for the host to enumerate the device. cdc_transport_close() is a no-op (USB stays open). cdc_transport_write() sends bytes using R_USB_Write() and blocks on a FreeRTOS semaphore until the USB completion interrupt fires. cdc_transport_read() pulls bytes from a FreeRTOS stream buffer that the USB receive interrupt fills asynchronously.
Other transport options supported by micro-ROS include UART (serial), UDP over Ethernet or Wi-Fi, and custom transports. USB-CDC was chosen for this project because the EK-RA6M5 has a dedicated USB Full Speed peripheral and because USB provides higher throughput than UART (up to 12 Mbps vs. typical 115200 baud).

2.3 The Client Library Stack

The micro-ROS client running on the microcontroller is composed of several layers, all compiled into a single static library called libmicroros.a:

Where to find it: micro_ros_renesas2estudio_component (Renesas build integration): https://github.com/micro-ROS/micro_ros_renesas2estudio_component — Micro-XRCE-DDS: https://github.com/eProsima/Micro-XRCE-DDS — micro-ROS project homepage: https://micro.ros.org/

- rclc (ROS Client Library for C) — the high-level API the application calls. Functions like rclc_publisher_init_default(), rclc_executor_spin_some().
- rcl (ROS Client Library) — the core library managing nodes, publishers, subscribers, timers, and execution.
- rcutils — utility functions (logging, allocators, error handling, string operations).
- rmw_microxrcedds — the middleware layer translating between rcl's internal representation and the XRCE-DDS wire format.
- Micro-XRCE-DDS Client — the serialization/deserialization engine producing binary frames for the transport.
- rosidl type support — generated code defining how each ROS 2 message type (sensor_msgs/Temperature, std_msgs/String, etc.) is serialized.

All are compiled into libmicroros.a, approximately 2–4 MB. The application links against it and calls its API.

Understanding this stack matters because when something goes wrong (and it will), the error messages reference specific layers. An "RCL_RET_BAD_ALLOC" error comes from rcl when malloc returns NULL. An "XRCE session not established" error comes from the XRCE-DDS client when it cannot reach the agent.

An "undefined reference to clock_gettime" linker error means rcutils needs a POSIX stub you have not provided. Knowing which layer owns which function helps you diagnose problems faster.
The library is built by cross-compiling all these layers together using the micro_ros_renesas2estudio_component build scripts. The output is a single .a file (static archive) plus a directory of header files. Section 6.3 covers this build process in detail — it is the most error-prone step of the entire project.

2.4 The Agent

The micro-ROS agent is a Linux process on the host computer. It bridges XRCE-DDS and full DDS:

- Opens the serial port (e.g., /dev/ttyACM0) and listens for XRCE-DDS frames.
- When the client creates a node, the agent creates a corresponding DDS participant on the ROS 2 network.
- When the client creates a publisher and topic, the agent creates a matching DDS topic and datawriter.
- When the client publishes a message, the agent deserializes it from XRCE-DDS, converts it to a full DDS message, and publishes it.
- For subscriptions, the reverse: the agent receives full DDS messages, serializes them to XRCE-DDS, and sends them to the microcontroller.

The agent also handles lifecycle management. When the microcontroller's XRCE-DDS session times out (board reset, USB disconnect, firmware crash), the agent automatically destroys the corresponding DDS entities and cleans up. When the client reconnects, the agent creates fresh entities. This means you can reset the board without restarting the agent — it handles reconnection automatically. You will see log messages like "Destroy participant" followed by "Create participant" when this happens.
The agent supports multiple simultaneous clients. If you had two microcontrollers connected via USB, you could run two agent instances on different serial ports, or use the agent's multi-serial mode. Each client maintains its own XRCE-DDS session independently.

Started with a single command:

```
ros2 run micro_ros_agent micro_ros_agent serial --dev /dev/ttyACM0
```

Where to find it: micro-ROS Agent repository: https://github.com/micro-ROS/micro-ROS-Agent

3. What Micro-ROS Does in This Project

3.1 The Without-Micro-ROS Baseline

Phase A (bare-metal USB version) demonstrates sensor monitoring without micro-ROS. The microcontroller reads three I2C sensors and prints formatted text over USB:

Room: 23.4C|74.1F | Hot: 24.1C|75.4F | Cold: 23.8C|74.8F | Hum: 45% | Door: Opened | Lux: 312

This works for basic monitoring but has fundamental limitations:

- Unstructured text. Every consumer must parse the string and handle formatting changes.
- Point-to-point. Only one program (the terminal) can receive data at a time.
- No timestamps, no units metadata, no quality-of-service guarantees, no standard schema.
- No discovery. You must manually configure the COM port and data format.
- Integration requires custom glue code for every consumer (logger, dashboard, controller, ML pipeline).

3.2 What Micro-ROS Changes

Phase B replaces text printing with micro-ROS. The same sensors are read, but structured ROS 2 messages are published to named topics:

Topic	Message Type	Data
/sensors/room_temp	sensor_msgs/msg/Temperature	Temperature in °C, with header timestamp and frame_id
/sensors/humidity	sensor_msgs/msg/RelativeHumidity	Humidity as 0.0–1.0 fraction, with header timestamp
/sensors/thermocouple_hot	sensor_msgs/msg/Temperature	MCP9600 hot junction temperature in °C
/sensors/thermocouple_cold	sensor_msgs/msg/Temperature	MCP9600 cold junction temperature in °C
/sensors/proximity	std_msgs/msg/UInt16	VCNL4010 raw proximity count (higher = closer)
/sensors/ambient_light	std_msgs/msg/UInt16	VCNL4010 raw ambient light count
/sensors/diagnostic	std_msgs/msg/String	Node status messages and error reports

Every Phase A limitation is now solved:

- Structured data. sensor_msgs/Temperature has defined fields. Any ROS 2 program in any language can deserialize it without parsing text.
- Multicast. Any number of nodes can subscribe simultaneously — logger, dashboard, and controller all receive the same stream.

- Discoverable. "ros2 topic list" shows all seven topics. "ros2 topic info" shows message types and publisher/subscriber counts.
- Timestamped. Each message carries publication time for time-series analysis and synchronization.
- Zero-effort integration. Log: "ros2 bag record /sensors/room_temp". Plot: open rqt_plot. Python: three lines of rclpy. Cloud: rosbridge node. None require knowledge of I2C or USB.

3.3 The Data Flow

Here is exactly what happens when the RA6M5 publishes a temperature reading:

- The 1 Hz timer callback fires inside the FreeRTOS task on the RA6M5.
- sensors_read_sht3x() performs an I2C transaction to the SHT3x at address 0x44 and returns temperature in °C.
- The firmware populates a sensor_msgs/Temperature struct: temperature field, header timestamp (from FreeRTOS tick count), frame_id = "sht3x".
- rcl_publish() serializes the struct into a CDR byte buffer via rosidl type support.
- The buffer is wrapped in an XRCE-DDS frame by the Micro-XRCE-DDS client.
- The frame is passed to cdc_transport_write(), which calls R_USB_Write() to send bytes over USB.
- Bytes travel over USB to /dev/ttyACM0 on the host.
- The micro-ROS agent deserializes the XRCE-DDS frame back into a sensor_msgs/Temperature and publishes through DDS.
- Any subscribed ROS 2 node receives the message. It is indistinguishable from a native Linux publisher.

This entire chain — from I2C sensor read to DDS publication — completes in under 10 milliseconds.

To verify this chain is working, open three terminals on the host: Terminal 1 runs the agent (ros2 run micro_ros_agent ...), Terminal 2 runs "ros2 topic list" to confirm all seven topics appear, and Terminal 3 runs "ros2 topic echo /sensors/room_temp" to see the actual sensor_msgs/Temperature messages arriving. Each message will show a header with a timestamp, a temperature value in degrees Celsius, and a variance field (set to 0.0 since we are not computing measurement uncertainty).
Expected output from ros2 topic list:

```
/sensors/room_temp
/sensors/room_temp_f
/sensors/humidity
/sensors/hot_junction
/sensors/cold_junction
/sensors/proximity
/sensors/ambient_light
```

Expected output from ros2 topic echo /sensors/room_temp:

```
header:
  stamp:
    sec: 45
```

```
    nanosec: 500000000
  frame_id: sht3x
temperature: 24.86
variance: 0.0
```

If topics do not appear: (1) Check that the agent printed "Create participant" and shows 7 datawriters. If it shows 0, the board is not sending data — check USB connection and firmware. (2) If the agent shows participants but ros2 topic list is empty, this is the DDS multicast issue described in Section 12.4 — switch to native Linux. (3) If topics appear but echo shows no data, the publisher might be crashing during message serialization — check breadcrumb values and error variables on the board.

Additional useful commands for debugging: "ros2 topic info /sensors/room_temp" shows the message type and publisher/subscriber count. "ros2 topic hz /sensors/room_temp" measures the actual publication rate (should be ~1 Hz). "ros2 bag record -a" records all topics to a rosbag file for offline analysis.

The seven topics published by this project are: /sensors/room_temp (SHT3x temperature), /sensors/room_temp_f (SHT3x temperature in Fahrenheit), /sensors/humidity (SHT3x relative humidity), /sensors/hot_junction (MCP9600 thermocouple hot junction), /sensors/cold_junction (MCP9600 cold junction / ambient), /sensors/proximity (VCNL4010 proximity count), and /sensors/ambient_light (VCNL4010 ambient light count). All use sensor_msgs/Temperature except proximity and ambient light which use the same message type with the temperature field repurposed for the sensor reading.

3.4 Micro-ROS vs. Alternatives

Approach	Pros	Cons
Raw serial (Phase A)	Simple, no dependencies, fast to implement	Unstructured, point-to-point, no discovery, no timestamps, custom parsers needed
MQTT	Lightweight, IoT-friendly, cloud-native	Not native to ROS 2, no standardized message types, requires a bridge
rosserial (ROS 1)	Proven, well-documented	Deprecated, ROS 1 only, no DDS integration
Custom bridge	Full control	Must write and maintain serialization, transport, and bridging code
micro-ROS	Native ROS 2, standard types, discovery, multi-subscriber, timestamped	Complex setup, ~200 KB heap, requires host-side agent

3.5 What Micro-ROS Requires from the Firmware

Micro-ROS imposes requirements that bare-metal does not:

- An RTOS — FreeRTOS in this project. The micro-ROS node runs as a task with a 40,000-byte stack.
- A large heap — 200,000 bytes (configTOTAL_HEAP_SIZE). Micro-ROS allocates 80–120 KB during initialization.

This is one of the most insidious bugs in the project (Bug #2, Section 8.2). FreeRTOS's heap size is defined in FreeRTOSConfig.h with a #ifndef guard, meaning the header's value takes priority over any -D compiler flag. If you try to set the heap size via Project Properties → Compiler → Preprocessor Defines → -DconfigTOTAL_HEAP_SIZE=200000, it will be silently ignored because the generated header already defines it as 1024. You must edit the file directly.

Open ra_cfg/aws/FreeRTOSConfig.h. Find the line "#define configTOTAL_HEAP_SIZE" and change the value to 200000. Warning: every time you click "Generate Project Content" in the FSP configurator, this file may be regenerated with the default value. After every regeneration, check this file and fix the value if needed. A good practice is to add a comment "/* MANUALLY SET - DO NOT REGENERATE */" next to the line.

Why 200,000 bytes? Micro-ROS allocates approximately 80-120 KB during initialization for DDS session state, topic registrations, message buffers, and internal data structures. The remaining heap is used during runtime for message serialization buffers. With 200 KB, there is enough headroom for seven publishers plus the XRCE-DDS session. If you add more publishers or subscribers, you may need to increase this value. You can monitor heap usage at runtime by calling xPortGetFreeHeapSize() and printing the result.

To verify the heap size took effect: build the project, then check the .map file (in the Debug directory). Look for the .bss section size — it should be approximately 200 KB + FreeRTOS overhead + other BSS variables, totaling roughly 260-270 KB. If .bss is only ~10 KB, the heap size change did not take effect.

- A working malloc — overridden globally to route through pvPortMalloc (freertos_malloc_override.c).
- POSIX stubs — clock_gettime, gettimeofday, usleep, isatty, fprintf (microros_stubs.c).
- A custom transport — USB-CDC open/close/write/read callbacks (microros_transports.c).
- A cross-compiled library — libmicroros.a built for Cortex-M33 with hardware FPU.

Each of these requirements represents a significant piece of work. The RTOS adds task scheduling, semaphores, and stream buffers but also introduces complexity around task priorities, stack sizes, and heap management. The 200 KB heap must be carefully sized — too small and micro-ROS initialization fails silently; too large and you run out of RAM for task stacks and sensor buffers. The malloc override is a single file but took days to diagnose because the symptom (rc=10 from rclc_support_init) gave no indication that the root cause was in the C standard library. The POSIX stubs are straightforward once you know which functions are needed, but discovering the list requires iterating through linker errors one at a time. The transport callbacks require understanding both the Renesas USB API and the micro-ROS transport interface. And the library cross-compilation is a multi-hour process with numerous failure modes (Section 6.3).

Part II: Complete Build Guide

4. Project Overview

4.1 Hardware Bill of Materials

Component	Description	Interface	I2C Address
Renesas EK-RA6M5	Dev board, ARM Cortex-M33, 512 KB SRAM	USB Micro-B (J11 debug, J12 FS)	N/A
SHT3x	Temperature and humidity sensor (Sensirion)	I2C via QWIIC	0x44
MCP9600	Thermocouple interface (hot + cold junction)	I2C via QWIIC	0x67
VCNL4010	Proximity and ambient light sensor (Vishay)	I2C via QWIIC	0x13
USB Micro-B cable	Data cable (NOT power-only)	USB	N/A
QWIIC cables	4-pin JST connectors for I2C daisy-chain	I2C	N/A

4.2 Software Stack

Software	Version	Purpose
e² studio	Latest (Renesas IDE)	Project configuration, build, flash, debug
Renesas FSP	Bundled with e² studio	Hardware abstraction layer and drivers
clang / ATfE	21.1.1	C compiler toolchain
FreeRTOS	Bundled with FSP	RTOS (micro-ROS version only)
micro-ROS (libmicroros.a)	Humble	ROS 2 client library for microcontrollers
ROS 2 Humble	Humble Hawksbill	Host-side ROS 2 framework
micro_ros_agent	Built from source	Serial-to-DDS bridge on host
WSL2 (Ubuntu 22.04)	Jammy Jellyfish	Linux environment on Windows
usbipd-win	Latest	USB passthrough from Windows to WSL2
Podman	3.4.4+	Container runtime for deployment
TeraTerm	Latest	Serial terminal (bare-metal testing)

Where to find it: e² studio IDE: https://www.renesas.com/en/software-tool/e-studio — Renesas FSP (GitHub): https://github.com/renesas/fsp — EK-RA6M5 board page: https://www.renesas.com/en/products/microcontrollers-microprocessors/ra-cortex-m-mcus/ek-ra6m5-evaluation-kit-ra6m5-mcu-group

4.3 I2C Wiring

All three sensors connect via QWIIC daisy-chain on I2C channel 2. Pin assignments in FSP:

- SDA: P414
- SCL: P415
- I2C mode: Standard rate, 7-bit addressing

QWIIC connectors handle pull-up resistors internally. No external pull-ups needed.

The QWIIC (Sparkfun) / STEMMA QT (Adafruit) connector system uses a standardized 4-pin JST SH connector carrying GND, 3.3V, SDA, and SCL. You can daisy-chain multiple sensors by connecting a QWIIC cable from the first sensor's output port to the second sensor's input port, and so on. Each sensor has a unique I2C address, so they share the same bus without conflict.
The three sensors in this project use the following I2C addresses: SHT3x temperature/humidity at 0x44 (the default, can be changed to 0x45 by connecting the ADDR pin to VDD), MCP9600 thermocouple amplifier at 0x67 (configurable via address pins A0/A1), and VCNL4010 proximity/ambient light at 0x13 (fixed, cannot be changed). You can verify these addresses are responding using an I2C scanner — write a short firmware that tries reading from each address and reports success/failure.
Physical wiring: connect the EK-RA6M5's PMOD/Grove connector or use jumper wires from the board's P414 (SDA) and P415 (SCL) pins to the first sensor's QWIIC input. If your board does not have a QWIIC connector, use a QWIIC-to-breadboard cable (Sparkfun part BOB-14425) to break out the four lines to individual pins. The board supplies 3.3V on its PMOD connector; connect this to the sensor chain's VCC. Connect GND to GND. The total cable length should stay under 30 cm for reliable I2C at standard rate (100 kHz).

4.3.1 Sensor Details

SHT3x (Temperature + Humidity): Sensirion SHT30/SHT31/SHT35, I2C address 0x44. Measurement command 0x2C06 triggers a single high-repeatability measurement. Response is 6 bytes: [temp MSB, temp LSB, temp CRC, hum MSB, hum LSB, hum CRC]. CRC bytes (indices 2 and 5) are ignored in this project. Temperature formula: T(°C) = -45 + 175 × (rawTemp / 65535). Humidity formula: RH = rawHum / 65535. Wait at least 15ms after sending the measurement command before reading (this project uses 20ms). Datasheet: https://sensirion.com/products/catalog/SHT30-DIS-B/
MCP9600 (Thermocouple Amplifier): Microchip MCP9600, I2C address 0x67. Reads Type-K thermocouple temperature. Hot junction register is 0x00, cold junction (ambient) is 0x02. Each returns 2 bytes as a signed 16-bit integer. Resolution is 0.0625°C per LSB (divide raw value by 16). Supports -200°C to +1372°C (Type-K range). The cold junction reading is useful as a second ambient temperature measurement. Datasheet: https://www.microchip.com/en-us/product/MCP9600
VCNL4010 (Proximity + Ambient Light): Vishay VCNL4010, I2C address 0x13 (fixed). Initialization requires writing IR LED current (register 0x82, value 20 = 200mA) and ambient light config (register 0x84, value 0x9D = continuous conversion, 10 samples/s, auto-offset). To trigger an on-demand measurement, write 0x18 to register 0x80, wait 100ms, then read proximity from register 0x87 (2 bytes, unsigned 16-bit) and ambient light from register 0x85 (2 bytes, unsigned 16-bit). Proximity values above ~10,000 indicate an object within a few centimeters (used as a door open/closed detector in this project). Datasheet: https://www.vishay.com/docs/83462/vcnl4010.pdf

5. Phase A: Bare-Metal USB Version

Standalone firmware that reads all three sensors and prints formatted text over USB-CDC to a serial terminal. No RTOS, no ROS. Hardware validation.

5.1 Create the e² Studio Project

1. File → New → Renesas C/C++ Project → Renesas RA.
2. Board: EK-RA6M5.
3. Template: "Bare Metal – Minimal". Name: "Sensor_Project_USB_Version". Finish.

When you click Finish, e² studio generates a project with the Renesas FSP (Flexible Software Package) framework. FSP provides HAL (Hardware Abstraction Layer) drivers for every peripheral on the RA6M5. The generated project includes ra/, ra_gen/, and ra_cfg/ directories containing the driver source, generated configuration, and editable configuration headers respectively. The main entry point is src/hal_entry.c.
Before writing any code, you need to configure peripherals using the FSP configurator. Open the configuration.xml file in the project root — this launches a graphical editor with tabs for BSP, Clocks, Pins, Stacks, and Components. The Stacks tab is where you add peripheral drivers.

5.2 Configure FSP Peripherals

5.2.1 I2C Master

1. Stacks tab → New Stack → Connectivity → I2C Master (r_iic_master).
2. Properties: Channel 2, Standard rate, Slave Address 0x44, 7-Bit, Callback: i2c_master_callback, SDA: P414, SCL: P415.

In the FSP configurator, click "New Stack" and search for "I2C Master." Select "I2C Master (r_iic_master)." In the Properties panel on the right, set: Name = g_i2c_master0, Channel = 2, Rate = Standard, Slave Address = 0x44 (this is the SHT3x; you will change it in code when talking to other sensors), Address Mode = 7-Bit, Callback = i2c_master_callback. Under the Pins tab, verify that P414 is assigned to IIC2_SDA and P415 to IIC2_SCL. If the pins show as "Not Used," click on them and select the correct peripheral function.
Channel 2 is important — the EK-RA6M5 has multiple I2C channels, and the pins P414/P415 are specifically routed to channel 2 in the chip's pin mux. Using a different channel with these pins will not work. The slave address set here is just a default; the code calls R_IIC_MASTER_SlaveAddressSet() before each transaction to switch between the three sensor addresses.

5.2.2 USB PCDC

1. Stacks tab → New Stack → Connectivity → USB PCDC (r_usb_pcdc).
2. Properties: USB Mode = Peri, Speed = Full Speed, Module = USB_IP0.
3. Generate Project Content (Ctrl+Shift+G).

After adding the USB PCDC stack, you must click "Generate Project Content" (the green arrow button, or Ctrl+Shift+G). This regenerates ra_gen/hal_data.c and hal_data.h with the control structures and

configuration for the peripherals you configured. If you skip this step, the compiler will report undefined references to g_basic0_ctrl, g_basic0_cfg, g_i2c_master0_ctrl, etc.
The USB PCDC stack adds a significant amount of generated code. You will see g_basic0 (the USB instance) with associated control and config structures. The USB peripheral uses the RA6M5's dedicated USB Full Speed controller (USB_IP0). The board has two USB ports: J11 (debug/programming via the on-board debugger) and J12 (the USB Full Speed peripheral). Your sensor data goes through J12. You will need two USB cables: one for J11 (always connected for debugging/flashing) and one for J12 (the data connection to the host).

5.2.3 Pins

Pins tab: verify P414 = IIC2_SDA, P415 = IIC2_SCL, USB pins auto-configured.

5.3 Application Code: hal_entry.c

The entire application lives in src/hal_entry.c:

This bare-metal version has no RTOS — the code runs in a single infinite while(1) loop. Understanding this version first is important because it validates that all three sensors are wired correctly and responding on I2C before adding the complexity of FreeRTOS and micro-ROS. If sensors do not work in Phase A, they will not work in Phase B, and you will waste time debugging micro-ROS when the real problem is a wiring issue.
The key architectural insight in this code is the USB polling requirement. Unlike UART, USB requires active participation from the firmware to maintain the connection. The R_USB_EventGet() call processes the USB state machine — without it, the USB enumeration never completes and the host never sees the device. This call must happen frequently (at least every few milliseconds). If you put a long delay in the main loop, USB will stop working. This is the most common pitfall with bare-metal USB on Renesas.

```
#include "hal_data.h"
#include <stdio.h>
#include <string.h>

#define SHT3X_ADDR    0x44
#define MCP9600_ADDR  0x67
#define VCNL4010_ADDR 0x13

volatile bool i2c_done = false;

void i2c_master_callback(i2c_master_callback_args_t *p_args) {
   if (p_args->event == I2C_MASTER_EVENT_TX_COMPLETE ||
      p_args->event == I2C_MASTER_EVENT_RX_COMPLETE) {
      i2c_done = true;
   }
}

void i2c_write_blocking(uint8_t dev_addr, uint8_t *data, uint32_t bytes) {
```

```
    R_IIC_MASTER_SlaveAddressSet(&g_i2c_master0_ctrl, dev_addr, I2C_MASTER_ADDR_MODE_7BIT);
    i2c_done = false;
    R_IIC_MASTER_Write(&g_i2c_master0_ctrl, data, bytes, false);
    while (!i2c_done);
}

void i2c_read_blocking(uint8_t dev_addr, uint8_t *data, uint32_t bytes) {
    R_IIC_MASTER_SlaveAddressSet(&g_i2c_master0_ctrl, dev_addr, I2C_MASTER_ADDR_MODE_7BIT);
    i2c_done = false;
    R_IIC_MASTER_Read(&g_i2c_master0_ctrl, data, bytes, false);
    while (!i2c_done);
}

void hal_entry(void) {
    R_USB_Open(&g_basic0_ctrl, &g_basic0_cfg);
    R_IIC_MASTER_Open(&g_i2c_master0_ctrl, &g_i2c_master0_cfg);

    char print_buffer[250];
    uint8_t setup_data[2];

    uint8_t sht_cmd[2] = {0x2C, 0x06};
    uint8_t sht_data[6];
    uint8_t mcp_reg;
    uint8_t mcp_data[2];
    uint8_t vcnl_cmd[2];
    uint8_t vcnl_data[2];

    // Wake up the VCNL4010 Lux sensor
    setup_data[0] = 0x82; setup_data[1] = 20;
    i2c_write_blocking(VCNL4010_ADDR, setup_data, 2);
    setup_data[0] = 0x84; setup_data[1] = 0x9D;
    i2c_write_blocking(VCNL4010_ADDR, setup_data, 2);

    uint32_t loop_counter = 0;
    bool usb_connected = false;

    while (1) {
        usb_event_info_t event_info;
        usb_status_t event;

        // --- THE MAGIC FIX ---
        // This MUST be called constantly in bare-metal to process the USB state machine
        fsp_err_t err = R_USB_EventGet(&event_info, &event);

        if (err == FSP_SUCCESS) {
            if (event == USB_STATUS_CONFIGURED) {
                usb_connected = true;
            } else if (event == USB_STATUS_DETACH) {
```

```
      usb_connected = false;
    }
}

if (usb_connected) {
    loop_counter++;

    // Read sensors roughly every 1000 loops (~1 second)
    if (loop_counter >= 1000) {
        loop_counter = 0;

        // 1. SHT3x
        i2c_write_blocking(SHT3X_ADDR, sht_cmd, 2);
        R_BSP_SoftwareDelay(20, BSP_DELAY_UNITS_MILLISECONDS);
        i2c_read_blocking(SHT3X_ADDR, sht_data, 6);
        uint16_t rawTemp = (uint16_t)((sht_data[0] << 8) | sht_data[1]);
        uint16_t rawHum  = (uint16_t)((sht_data[3] << 8) | sht_data[4]);
        float roomC = -45.0f + (175.0f * ((float)rawTemp / 65535.0f));
        float roomF = (roomC * 1.8f) + 32.0f;
        float humid = 100.0f * ((float)rawHum / 65535.0f);

        // 2. MCP9600
        mcp_reg = 0x00;
        i2c_write_blocking(MCP9600_ADDR, &mcp_reg, 1);
        i2c_read_blocking(MCP9600_ADDR, mcp_data, 2);
        int16_t rawHot = (int16_t)((mcp_data[0] << 8) | mcp_data[1]);
        float hotC = (float)rawHot * 0.0625f;
        float hotF = (hotC * 1.8f) + 32.0f;

        mcp_reg = 0x02;
        i2c_write_blocking(MCP9600_ADDR, &mcp_reg, 1);
        i2c_read_blocking(MCP9600_ADDR, mcp_data, 2);
        int16_t rawCold = (int16_t)((mcp_data[0] << 8) | mcp_data[1]);
        float coldC = (float)rawCold * 0.0625f;
        float coldF = (coldC * 1.8f) + 32.0f;

        // 3. VCNL4010
        vcnl_cmd[0] = 0x80; vcnl_cmd[1] = 0x18;
        i2c_write_blocking(VCNL4010_ADDR, vcnl_cmd, 2);
        R_BSP_SoftwareDelay(100, BSP_DELAY_UNITS_MILLISECONDS);

        vcnl_cmd[0] = 0x87;
        i2c_write_blocking(VCNL4010_ADDR, vcnl_cmd, 1);
        i2c_read_blocking(VCNL4010_ADDR, vcnl_data, 2);
        uint16_t prox = (uint16_t)((vcnl_data[0] << 8) | vcnl_data[1]);

        vcnl_cmd[0] = 0x85;
        i2c_write_blocking(VCNL4010_ADDR, vcnl_cmd, 1);
```

```
            i2c_read_blocking(VCNL4010_ADDR, vcnl_data, 2);
            uint16_t lux = (uint16_t)((vcnl_data[0] << 8) | vcnl_data[1]);

            char* doorStatus = (prox >= 10000) ? "Closed" : "Opened";

            // 4. Print
            sprintf(print_buffer,
               "Room: %.1fC|%.1fF | Hot: %.1fC|%.1fF | Cold: %.1fC|%.1fF | Hum: %.0f%% | Door: %s | Lux:
%u\r\n",
               roomC, roomF, hotC, hotF, coldC, coldF, humid, doorStatus, lux);

            R_USB_Write(&g_basic0_ctrl, (uint8_t *)print_buffer, (uint32_t)strlen(print_buffer),
USB_CLASS_PCDC);
         }
      }

      // A tiny 1ms delay allows the loop to spin fast enough to keep USB alive
      R_BSP_SoftwareDelay(1, BSP_DELAY_UNITS_MILLISECONDS);
   }
}
```

CRITICAL: USB Polling
R_USB_EventGet() in the while(1) loop is essential. In bare-metal mode, the USB state machine only advances when you call this function. Without it, the board never enumerates.

5.4 Build, Flash, and Verify

1. Build (Ctrl+B). Verify zero errors.
2. Connect J11 (debug port) to PC. Debug → Renesas GDB Hardware Debugging. Flash is automatic.
3. Play (F8). Connect second USB cable to J12 (data port).
4. TeraTerm: Serial, select COM port. Output appears once per second.

Step-by-step build and flash procedure: (1) Click the hammer icon or press Ctrl+B to build. The Output console should show "Build Finished. 0 errors, 0 warnings." If you see linker errors about undefined symbols, you likely forgot to Generate Project Content after adding stacks. (2) Connect a USB cable from your PC to the board's J11 (debug) port. The on-board debugger (J-Link OB) should enumerate. (3) Click Run → Debug Configurations → Renesas GDB Hardware Debugging. Create a new configuration if one does not exist. The default settings work. Click Debug. The firmware is flashed automatically, and execution pauses at the entry point. (4) Press F8 (Resume) or click the green Play button. (5) Connect a second USB cable from your PC to J12 (the Full Speed USB port). (6) Open your serial terminal (TeraTerm, PuTTY, minicom, or screen). Select the new COM port that appeared. Baud rate does not matter for USB-CDC — it is ignored. (7) You should see sensor readings scrolling once per second.
If no COM port appears after connecting J12: check that the firmware is actually running (not paused in the debugger). If TeraTerm shows the port but no data: the firmware might be stuck in the USB

enumeration — press the board's reset button while J12 is connected. If you see garbled text: this should not happen with USB-CDC (it is a digital protocol), but verify you selected the correct COM port (not the debug port).
Expected output format:
Room: 23.4C|74.1F | Hot: 24.1C|75.4F | Cold: 23.8C|74.8F | Hum: 45% | Door: Opened | Lux: 312
Verify that all five readings change when you interact with the sensors: breathe on the SHT3x to raise humidity, touch the MCP9600 thermocouple to raise the hot junction temperature, and wave your hand near the VCNL4010 to change the proximity reading and toggle the door status. If any reading is stuck at 0 or shows unreasonable values, that sensor has a wiring issue — go back and check the I2C connections.

Known Issue: Slow USB Enumeration

On some Windows laptops, the board can take up to 15 minutes to appear as a COM device. This is a Windows USB driver issue. Try a different cable (many are power-only) or USB port.

6. Phase B: Micro-ROS Version

6.1 Create the Project

1. File → New → Renesas C/C++ Project → Renesas RA.
2. Board: EK-RA6M5. Toolchain: LLVM for ARM Embedded (clang) 21.1.1.
3. Template: FreeRTOS – Minimal. Name: "Sensor_Project_ROS". Finish.

Note the toolchain selection: LLVM for ARM Embedded (clang) 21.1.1, also called "ATfE" (ARM Toolchain for Embedded). This is different from Phase A which used the default GCC ARM toolchain. The clang toolchain is required because the micro-ROS library (libmicroros.a) was cross-compiled with clang, and mixing compilers for the library and application can cause ABI incompatibilities and linker errors.
Also note the template: FreeRTOS – Minimal, not Bare Metal. This template includes FreeRTOS kernel source and generates the initial task structure. Micro-ROS requires an RTOS because its internal state machines depend on blocking waits (semaphores, stream buffers) that only work inside an RTOS task context. Calling micro-ROS functions from bare-metal code (hal_entry or an ISR) will crash.

6.2 FSP Configuration

6.2.1 I2C Master

Same as Section 5.2.1 except callback = sensors_i2c_callback.

6.2.2 USB PCDC

Same as Section 5.2.2 except callback = cdc_usb_callback.

6.2.3 FreeRTOS Thread

1. Stacks tab: click existing thread. Name: sensor_node. Entry: new_thread0_entry.
2. Stack Size: 40000 bytes. Priority: 1. Generate Project Content.

The stack size of 40,000 bytes is critical. Micro-ROS uses deep call stacks during initialization (rclc_support_init → rcl_init → rmw_init → XRCE-DDS session creation), and also allocates temporary buffers on the stack during message serialization. The default FreeRTOS stack size of 1024 bytes causes an immediate stack overflow. 40,000 bytes provides comfortable headroom. If you see random crashes or hard faults during initialization, insufficient stack size is the first thing to check.
To set this in the FSP configurator: click on the existing thread in the Threads panel (it may be called "New Thread" by default). In the Properties panel, change Name to "sensor_node", Symbol to "sensor_node", Entry Function to "new_thread0_entry", and Stack Size to 40000. The entry function name must match exactly — FSP generates a stub file src/sensor_node_entry.c that calls your new_thread0_entry() function. You will create this function in your source file.

6.2.4 Critical: FreeRTOS Heap Size

After generating, open ra_cfg/aws/FreeRTOSConfig.h. Change:

```
#define configTOTAL_HEAP_SIZE (200000)
```

WARNING
The -D compiler flag CANNOT override this because of the #ifndef guard. Edit the file directly. FSP may reset it to 1024 on every Generate Project Content. Always verify.

6.3 Building libmicroros.a

6.3.1 What Did NOT Work

- library_generation.sh pulls bleeding-edge source with outdated rules → cascading failures

Where to find it: Library build scripts: https://github.com/micro-ROS/micro_ros_renesas2estudio_component/tree/humble/library_generation — XRCE-DDS protocol specification: https://www.omg.org/spec/DDS-XRCE/

- Windows PATH parentheses cause Makefile syntax errors
- Each attempt: 40+ minutes before failing
- Even successful builds targeted wrong ARM (ARMv7-M instead of ARMv8-M): "conflicting CPU architectures 17/2"

The library generation script (library_generation.sh) clones the micro-ROS source code, configures it for the target platform, and cross-compiles everything into libmicroros.a. The script is designed for Docker containers running on Linux. When run inside WSL2, several things go wrong: (1) Windows PATH entries (like "C:\Program Files") contain parentheses that break Makefile syntax. (2) The script uses hardcoded ARM toolchain paths that may not match your installation. (3) If your system has the wrong version of cmake, colcon, or python3, the build fails with cryptic errors. (4) Most critically, the default ARM target flags often produce code for ARMv7-M (Cortex-M4) instead of ARMv8-M (Cortex-M33), leading to "conflicting CPU architectures 17/2" at link time — architecture 17 is ARMv8-M, 2 is ARMv7-M.

6.3.2 What Worked

- Clean PATH stripping Windows directories:

```
env -i HOME="$HOME" USER="$USER" PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin" bash -c '...'
```

- Correct flags: -mcpu=cortex-m33 -mthumb -mfpu=fpv5-sp-d16 -mfloat-abi=hard
- Extra package exclusions for C++-only packages

The clean PATH command strips all Windows directories from the environment so the build tools only see Linux paths. The ARM flags -mcpu=cortex-m33 -mthumb -mfpu=fpv5-sp-d16 -mfloat-abi=hard are essential: cortex-m33 selects the correct ARM architecture (ARMv8-M Mainline), mthumb generates Thumb-2 instructions, fpv5-sp-d16 selects the single-precision FPU variant present on the RA6M5, and mfloat-abi=hard passes floating-point values in FPU registers (required to match the FSP calling convention).
The C++-only package exclusions are needed because micro-ROS has optional C++ packages (like rclcpp, example_interfaces) that cannot be cross-compiled for bare-metal targets. If included, they cause build failures because the Cortex-M33 cross-compiler does not support C++ exceptions or RTTI by default. Excluding them reduces build time and avoids errors.
The entire build takes approximately 1.5 hours on a modern laptop. During this time, the script compiles several hundred source files across dozens of packages. The output is libmicroros.a (~2-4 MB) and a

directory of header files (include/). You only need to do this build once — the library is reusable across projects as long as you target the same CPU architecture and FPU configuration.

Successful build: ~1.5 hours.

6.3.3 Installing the Library

1. Copy libmicroros.a to: micro_ros_renesas2estudio_component/libmicroros/libmicroros.a
2. Copy include/ to: micro_ros_renesas2estudio_component/libmicroros/include/
3. Project Properties → Compiler → Includes: add include/ path.
4. Linker: add libmicroros.a path and "-lmicroros". Build to verify.

Detailed installation steps: (1) In your project directory, create the following path if it does not exist: micro_ros_renesas2estudio_component/libmicroros/. (2) Copy the built libmicroros.a into that directory. (3) Copy the entire include/ directory from the build output into the same location, so you have micro_ros_renesas2estudio_component/libmicroros/include/ with subdirectories like rcl/, rclc/, rmw/, etc. (4) In e² studio, right-click your project → Properties → C/C++ Build → Settings. Under the Compiler section, go to Includes and add the path to the include/ directory. (5) Under the Linker section, go to Libraries. Add the path to the directory containing libmicroros.a in the Library Search Path. Add "microros" (without lib prefix or .a suffix) to the Libraries list. (6) Build. You should see no "undefined reference" errors for micro-ROS functions. If you do, the library path or name is wrong.

6.4 Source Files

The micro-ROS version requires seven new source files beyond what FSP generates. Each file serves a specific purpose, and all are listed with complete code in Section 7. Here is what each file does and why it exists:

new_thread0_entry.c: The main application logic. Runs as a FreeRTOS task. Initializes sensors, sets up the micro-ROS transport, waits for the agent, creates a node with seven publishers, and enters a 1 Hz publish loop. This is the equivalent of hal_entry.c from Phase A but restructured for RTOS task context.

sensors.h / sensors.c: Sensor abstraction layer. Provides sensors_init(), sensors_read_sht3x(), sensors_read_mcp9600(), and sensors_read_vcnl4010(). Uses the same I2C transactions as Phase A but with FreeRTOS vTaskDelay() instead of bare-metal R_BSP_SoftwareDelay(), and adds timeout detection to prevent infinite hangs on I2C bus errors.

microros_transports.h / microros_transports.c: USB-CDC transport implementation for micro-ROS. The four required callbacks (open, close, write, read) plus the USB interrupt handler. Uses a FreeRTOS stream buffer for asynchronous receive and a binary semaphore for synchronous transmit.

microros_stubs.c: POSIX function stubs required by micro-ROS libraries. Provides clock_gettime() (using FreeRTOS tick count), gettimeofday(), usleep() (using vTaskDelay), isatty(), and fprintf() (stubbed to no-op). Each stub was added because the linker reported "undefined reference" during the build.

freertos_malloc_override.c: Global malloc/free/calloc/realloc overrides that route all dynamic memory allocation through FreeRTOS's pvPortMalloc/vPortFree. Required because micro-ROS's internal libraries call malloc() directly, bypassing the rcl-level allocator configuration.

usb_descriptor.c: USB device descriptors for the CDC class. Defines the vendor ID, product ID, device strings, and endpoint configuration. This file must match the USB PCDC configuration in FSP or the host will not enumerate the device correctly.

File	Purpose
new_thread0_entry.c	Main FreeRTOS task: micro-ROS init, 7 publishers, 1 Hz timer callback
sensors.c / sensors.h	Sensor abstraction: init, read with timeouts
microros_transports.c / .h	USB-CDC transport: stream buffers, semaphores
microros_stubs.c	POSIX shims: _exit, clock_gettime, usleep, fprintf, isatty
freertos_malloc_override.c	Global malloc/free/calloc/realloc → pvPortMalloc/vPortFree
usb_descriptor.c	CDC-ACM descriptors. VID 0x045B, PID 0x2013

7. Complete Source Code Listings (Micro-ROS Version)

Every source file in src/ reproduced verbatim. Copy-paste into your e² studio project.

7.1 new_thread0_entry.c

Main FreeRTOS task. Initializes sensors, registers USB-CDC transport, waits for agent, creates 7 publishers, spins 1 Hz timer.

```
#include "sensors.h"
#include "microros_transports.h"
#include <rcl/rcl.h>
#include <rcl/error_handling.h>
#include <rclc/rclc.h>
#include <rclc/executor.h>
#include <rmw_microros/rmw_microros.h>

#include <sensor_msgs/msg/temperature.h>
#include <sensor_msgs/msg/relative_humidity.h>
#include <std_msgs/msg/u_int16.h>
#include <std_msgs/msg/string.h>

#include <stdio.h>
#include <string.h>

#include "../ra_gen/new_thread0.h"
#include "../ra/aws/FreeRTOS/FreeRTOS/Source/include/FreeRTOS.h"
#include "../ra/aws/FreeRTOS/FreeRTOS/Source/include/task.h"

/* ---------------------------------------------------------------
 * Debug variables — check these in the debugger if something fails.
 *   g_breadcrumb:  which init step we reached
 *   g_last_rc:     return code from the last RCCHECK call
 * --------------------------------------------------------------- */
volatile int g_breadcrumb = 0;
volatile int g_last_rc = 0;
volatile int g_rc_check_num = 0;
char g_last_error_buf[256] = {0};

/* ---------------------------------------------------------------
 * micro-ROS state
 * --------------------------------------------------------------- */

static rcl_allocator_t      g_allocator;
static rclc_support_t       g_support;
static rcl_node_t           g_node;
```

```
static rclc_executor_t     g_executor;
static rcl_timer_t         g_timer;

static rcl_publisher_t pub_room_temp;
static rcl_publisher_t pub_humidity;
static rcl_publisher_t pub_hot;
static rcl_publisher_t pub_cold;
static rcl_publisher_t pub_prox;
static rcl_publisher_t pub_ambient;
static rcl_publisher_t pub_diag;

static sensor_msgs__msg__Temperature       msg_room_temp;
static sensor_msgs__msg__RelativeHumidity   msg_humidity;
static sensor_msgs__msg__Temperature       msg_hot;
static sensor_msgs__msg__Temperature       msg_cold;
static std_msgs__msg__UInt16               msg_prox;
static std_msgs__msg__UInt16               msg_ambient;
static std_msgs__msg__String               msg_diag;

static char diag_text[128];

/* ---------------------------------------------------------------
 * Helpers
 * --------------------------------------------------------------- */

#define RCCHECK(fn) do {                                        \
   g_rc_check_num++;                                            \
   rcl_ret_t _rc = (fn);                                        \
   g_last_rc = (int)_rc;                                        \
   if (_rc != RCL_RET_OK) {                                     \
      rcutils_error_string_t _err = rcl_get_error_string();     \
      strncpy(g_last_error_buf, _err.str, sizeof(g_last_error_buf) - 1);    \
      g_last_error_buf[sizeof(g_last_error_buf) - 1] = '\0';    \
      g_breadcrumb = 999;                                       \
      while (1) { vTaskDelay(pdMS_TO_TICKS(1000)); }            \
   }                                                            \
} while (0)

static void diag_log(const char *text)
{
   size_t len = strlen(text);
   if (len >= sizeof(diag_text)) len = sizeof(diag_text) - 1;
   memcpy(diag_text, text, len);
   diag_text[len] = '\0';

   msg_diag.data.data = (char *)diag_text;
   msg_diag.data.size = len;
   msg_diag.data.capacity = sizeof(diag_text);
```

```
    (void)rcl_publish(&pub_diag, &msg_diag, NULL);
}

static void stamp_header(builtin_interfaces__msg__Time *stamp)
{
    int64_t now_ns = rmw_uros_epoch_nanos();
    stamp->sec     = (int32_t)(now_ns / 1000000000LL);
    stamp->nanosec = (uint32_t)(now_ns % 1000000000LL);
}

/* ---------------------------------------------------------------
 * Timer callback — runs every 1 second, reads all sensors, publishes
 * --------------------------------------------------------------- */

static void timer_callback(rcl_timer_t *timer, int64_t last_call_time)
{
    (void)timer;
    (void)last_call_time;

    float  tempC, humid, hotC, coldC;
    uint16_t prox, ambient;

    if (sensors_read_sht3x(&tempC, &humid)) {
        stamp_header(&msg_room_temp.header.stamp);
        msg_room_temp.temperature = tempC;
        msg_room_temp.variance    = 0.0;
        (void)rcl_publish(&pub_room_temp, &msg_room_temp, NULL);

        stamp_header(&msg_humidity.header.stamp);
        msg_humidity.relative_humidity = humid;
        msg_humidity.variance          = 0.0;
        (void)rcl_publish(&pub_humidity, &msg_humidity, NULL);
    } else {
        diag_log("SHT3x read failed");
    }

    if (sensors_read_mcp9600(&hotC, &coldC)) {
        stamp_header(&msg_hot.header.stamp);
        msg_hot.temperature = hotC;
        msg_hot.variance    = 0.0;
        (void)rcl_publish(&pub_hot, &msg_hot, NULL);

        stamp_header(&msg_cold.header.stamp);
        msg_cold.temperature = coldC;
        msg_cold.variance    = 0.0;
        (void)rcl_publish(&pub_cold, &msg_cold, NULL);
    } else {
```

```
        diag_log("MCP9600 read failed");
    }

    if (sensors_read_vcnl4010(&prox, &ambient)) {
        msg_prox.data = prox;
        (void)rcl_publish(&pub_prox, &msg_prox, NULL);

        msg_ambient.data = ambient;
        (void)rcl_publish(&pub_ambient, &msg_ambient, NULL);
    } else {
        diag_log("VCNL4010 read failed");
    }
}

/* ---------------------------------------------------------------
 * Thread entry
 * --------------------------------------------------------------- */

void new_thread0_entry(void *pvParameters)
{
    FSP_PARAMETER_NOT_USED(pvParameters);

    /* --- 1. Init sensors (I2C) --- */
    g_breadcrumb = 1;

    if (!sensors_init()) {
        g_breadcrumb = 100;
        while (1) { vTaskDelay(pdMS_TO_TICKS(1000)); }
    }

    /* --- 2. Register USB-CDC transport --- */
    g_breadcrumb = 2;

    rmw_uros_set_custom_transport(
        true,
        NULL,
        cdc_transport_open,
        cdc_transport_close,
        cdc_transport_write,
        cdc_transport_read
    );

    /* --- 3. Set up message frame IDs --- */
    g_breadcrumb = 3;

    static const char frame_sht[] = "sht3x";
    static const char frame_mcp[] = "mcp9600";
```

```
msg_room_temp.header.frame_id.data     = (char *)frame_sht;
msg_room_temp.header.frame_id.size     = sizeof(frame_sht) - 1;
msg_room_temp.header.frame_id.capacity = sizeof(frame_sht);

msg_humidity.header.frame_id.data      = (char *)frame_sht;
msg_humidity.header.frame_id.size      = sizeof(frame_sht) - 1;
msg_humidity.header.frame_id.capacity  = sizeof(frame_sht);

msg_hot.header.frame_id.data          = (char *)frame_mcp;
msg_hot.header.frame_id.size          = sizeof(frame_mcp) - 1;
msg_hot.header.frame_id.capacity      = sizeof(frame_mcp);

msg_cold.header.frame_id.data         = (char *)frame_mcp;
msg_cold.header.frame_id.size         = sizeof(frame_mcp) - 1;
msg_cold.header.frame_id.capacity     = sizeof(frame_mcp);

/* --- 4. Wait for micro-ROS agent --- */
g_breadcrumb = 4;
g_allocator = rcl_get_default_allocator();

while (rmw_uros_ping_agent(1000, 1) != RMW_RET_OK) {
  vTaskDelay(pdMS_TO_TICKS(500));
}

/* --- 5. Initialize micro-ROS support (logging, context, rmw) --- */
g_breadcrumb = 5;

RCCHECK(rclc_support_init(&g_support, 0, NULL, &g_allocator));

/* --- 6. Create node --- */
g_breadcrumb = 6;

RCCHECK(rclc_node_init_default(&g_node, "ra6m5_sensor_node", "", &g_support));

/* --- 7. Create publishers --- */
g_breadcrumb = 7;

RCCHECK(rclc_publisher_init_default(
  &pub_room_temp, &g_node,
  ROSIDL_GET_MSG_TYPE_SUPPORT(sensor_msgs, msg, Temperature),
  "/sensors/room_temp"));

RCCHECK(rclc_publisher_init_default(
  &pub_humidity, &g_node,
  ROSIDL_GET_MSG_TYPE_SUPPORT(sensor_msgs, msg, RelativeHumidity),
  "/sensors/humidity"));
```

```
  RCCHECK(rclc_publisher_init_default(
    &pub_hot, &g_node,
    ROSIDL_GET_MSG_TYPE_SUPPORT(sensor_msgs, msg, Temperature),
    "/sensors/thermocouple_hot"));

  RCCHECK(rclc_publisher_init_default(
    &pub_cold, &g_node,
    ROSIDL_GET_MSG_TYPE_SUPPORT(sensor_msgs, msg, Temperature),
    "/sensors/thermocouple_cold"));

  RCCHECK(rclc_publisher_init_default(
    &pub_prox, &g_node,
    ROSIDL_GET_MSG_TYPE_SUPPORT(std_msgs, msg, UInt16),
    "/sensors/proximity"));

  RCCHECK(rclc_publisher_init_default(
    &pub_ambient, &g_node,
    ROSIDL_GET_MSG_TYPE_SUPPORT(std_msgs, msg, UInt16),
    "/sensors/ambient_light"));

  RCCHECK(rclc_publisher_init_default(
    &pub_diag, &g_node,
    ROSIDL_GET_MSG_TYPE_SUPPORT(std_msgs, msg, String),
    "/sensors/diagnostic"));

  /* --- 8. Create timer + executor --- */
  g_breadcrumb = 8;

  RCCHECK(rclc_timer_init_default2(
    &g_timer, &g_support,
    RCL_MS_TO_NS(1000),
    timer_callback,
    true));

  RCCHECK(rclc_executor_init(&g_executor, &g_support.context, 1, &g_allocator));
  RCCHECK(rclc_executor_add_timer(&g_executor, &g_timer));

  /* --- 9. Running --- */
  g_breadcrumb = 9;

  diag_log("ra6m5_sensor_node initialized");

  while (1) {
    rclc_executor_spin_some(&g_executor, RCL_MS_TO_NS(100));
    vTaskDelay(pdMS_TO_TICKS(10));
  }
}
```

7.2 sensors.h

```
#ifndef SENSORS_H
#define SENSORS_H

#include <stdint.h>
#include <stdbool.h>

/* Call once before any read. Opens I2C, wakes VCNL4010. */
bool sensors_init(void);

/* Read SHT3x temperature (°C) and humidity (0.0–1.0 fraction). */
bool sensors_read_sht3x(float *temp_c, float *humidity);

/* Read MCP9600 hot-junction and cold-junction temperatures (°C). */
bool sensors_read_mcp9600(float *hot_c, float *cold_c);

/* Read VCNL4010 raw proximity and ambient light counts. */
bool sensors_read_vcnl4010(uint16_t *prox, uint16_t *ambient);

#endif
```

7.3 sensors.c

Sensor abstraction. Uses vTaskDelay and 100ms I2C timeouts to avoid hanging on bus errors.

```
#include "sensors.h"

#include "../ra_gen/hal_data.h"
#include "../ra/aws/FreeRTOS/FreeRTOS/Source/include/FreeRTOS.h"
#include "../ra/aws/FreeRTOS/FreeRTOS/Source/include/task.h"

#define SHT3X_ADDR    0x44
#define MCP9600_ADDR  0x67
#define VCNL4010_ADDR 0x13

static volatile bool g_i2c_done = false;
static volatile bool g_i2c_error = false;

/* Called by FSP I2C driver on transfer complete. Wire this up in FSP
 * configurator as the callback for g_i2c_master0 (see FSP setup step). */
void sensors_i2c_callback(i2c_master_callback_args_t *p_args)
{
```

```
    if (p_args->event == I2C_MASTER_EVENT_TX_COMPLETE ||
        p_args->event == I2C_MASTER_EVENT_RX_COMPLETE) {
        g_i2c_done = true;
    } else {
        g_i2c_error = true;
        g_i2c_done = true;
    }
}

static bool i2c_write(uint8_t dev_addr, const uint8_t *data, uint32_t len)
{
    if (R_IIC_MASTER_SlaveAddressSet(&g_i2c_master0_ctrl, dev_addr,
                                     I2C_MASTER_ADDR_MODE_7BIT) != FSP_SUCCESS)
        return false;
    g_i2c_done = false;
    g_i2c_error = false;
    if (R_IIC_MASTER_Write(&g_i2c_master0_ctrl, (uint8_t *)data, len, false)
        != FSP_SUCCESS)
        return false;
    /* Wait with a timeout so we don't hang forever on bus errors. */
    uint32_t timeout = 100;  /* 100 ms */
    while (!g_i2c_done && timeout--) {
        vTaskDelay(pdMS_TO_TICKS(1));
    }
    return g_i2c_done && !g_i2c_error;
}

static bool i2c_read(uint8_t dev_addr, uint8_t *data, uint32_t len)
{
    if (R_IIC_MASTER_SlaveAddressSet(&g_i2c_master0_ctrl, dev_addr,
                                     I2C_MASTER_ADDR_MODE_7BIT) != FSP_SUCCESS)
        return false;
    g_i2c_done = false;
    g_i2c_error = false;
    if (R_IIC_MASTER_Read(&g_i2c_master0_ctrl, data, len, false) != FSP_SUCCESS)
        return false;
    uint32_t timeout = 100;
    while (!g_i2c_done && timeout--) {
        vTaskDelay(pdMS_TO_TICKS(1));
    }
    return g_i2c_done && !g_i2c_error;
}

bool sensors_init(void)
{
    if (R_IIC_MASTER_Open(&g_i2c_master0_ctrl, &g_i2c_master0_cfg) != FSP_SUCCESS)
        return false;
```

```
    /* Wake up the VCNL4010 ambient light / proximity sensor. */
    uint8_t setup[2];
    setup[0] = 0x82; setup[1] = 20;       /* IR LED current */
    if (!i2c_write(VCNL4010_ADDR, setup, 2)) return false;
    setup[0] = 0x84; setup[1] = 0x9D;     /* ambient light config */
    if (!i2c_write(VCNL4010_ADDR, setup, 2)) return false;

    return true;
}

bool sensors_read_sht3x(float *temp_c, float *humidity)
{
    uint8_t cmd[2] = {0x2C, 0x06};
    uint8_t raw[6];
    if (!i2c_write(SHT3X_ADDR, cmd, 2)) return false;
    vTaskDelay(pdMS_TO_TICKS(20));
    if (!i2c_read(SHT3X_ADDR, raw, 6)) return false;

    uint16_t rawTemp = (uint16_t)((raw[0] << 8) | raw[1]);
    uint16_t rawHum  = (uint16_t)((raw[3] << 8) | raw[4]);
    *temp_c   = -45.0f + (175.0f * ((float)rawTemp / 65535.0f));
    *humidity = (float)rawHum / 65535.0f;  /* 0.0–1.0 fraction */
    return true;
}

bool sensors_read_mcp9600(float *hot_c, float *cold_c)
{
    uint8_t reg;
    uint8_t raw[2];

    reg = 0x00;
    if (!i2c_write(MCP9600_ADDR, &reg, 1)) return false;
    if (!i2c_read(MCP9600_ADDR, raw, 2)) return false;
    int16_t rawHot = (int16_t)((raw[0] << 8) | raw[1]);
    *hot_c = (float)rawHot * 0.0625f;

    reg = 0x02;
    if (!i2c_write(MCP9600_ADDR, &reg, 1)) return false;
    if (!i2c_read(MCP9600_ADDR, raw, 2)) return false;
    int16_t rawCold = (int16_t)((raw[0] << 8) | raw[1]);
    *cold_c = (float)rawCold * 0.0625f;

    return true;
}

bool sensors_read_vcnl4010(uint16_t *prox, uint16_t *ambient)
{
    uint8_t cmd[2];
```

```
    uint8_t raw[2];

    /* Trigger a proximity + ambient on-demand measurement. */
    cmd[0] = 0x80; cmd[1] = 0x18;
    if (!i2c_write(VCNL4010_ADDR, cmd, 2)) return false;
    vTaskDelay(pdMS_TO_TICKS(100));

    cmd[0] = 0x87;
    if (!i2c_write(VCNL4010_ADDR, cmd, 1)) return false;
    if (!i2c_read(VCNL4010_ADDR, raw, 2)) return false;
    *prox = (uint16_t)((raw[0] << 8) | raw[1]);

    cmd[0] = 0x85;
    if (!i2c_write(VCNL4010_ADDR, cmd, 1)) return false;
    if (!i2c_read(VCNL4010_ADDR, raw, 2)) return false;
    *ambient = (uint16_t)((raw[0] << 8) | raw[1]);

    return true;
}
```

7.4 microros_transports.h

```
#ifndef MICROROS_TRANSPORTS_H
#define MICROROS_TRANSPORTS_H

#include <uxr/client/transport.h>
#include <stdbool.h>
#include <stdint.h>
#include <stddef.h>

#include "../ra_gen/hal_data.h"
#include "../ra/fsp/inc/api/r_usb_basic_api.h"
#include "../ra/fsp/inc/instances/r_usb_basic.h"

bool cdc_transport_open(struct uxrCustomTransport *transport);
bool cdc_transport_close(struct uxrCustomTransport *transport);
size_t cdc_transport_write(struct uxrCustomTransport *transport,
                    const uint8_t *buf, size_t len, uint8_t *err);
size_t cdc_transport_read(struct uxrCustomTransport *transport,
                   uint8_t *buf, size_t len, int timeout, uint8_t *err);

/* Called from FSP USB PCDC callback when data arrives or a write completes.
 * Wire this up as the callback for g_basic0 in FSP (USB stack's p_callback). */
void cdc_usb_callback(usb_event_info_t *p_event_info, usb_hdl_t handle,
                  usb_onoff_t on_off);

#endif
```

7.5 microros_transports.c

USB-CDC transport. FreeRTOS stream buffer for RX, binary semaphore for TX. Note USB_STATUS_SUSPEND is a no-op — Windows suspends during idle.

```
#include "microros_transports.h"
#include <string.h>
#include "../ra/aws/FreeRTOS/FreeRTOS/Source/include/FreeRTOS.h"
#include "../ra/aws/FreeRTOS/FreeRTOS/Source/include/semphr.h"
#include "../ra/aws/FreeRTOS/FreeRTOS/Source/include/stream_buffer.h"

extern usb_instance_ctrl_t g_basic0_ctrl;
extern const usb_cfg_t     g_basic0_cfg;

#define RX_BUFFER_SIZE      2048
```

```
#define RX_TRIGGER_LEVEL    1
#define WRITE_TIMEOUT_MS    1000

static StreamBufferHandle_t g_rx_stream = NULL;
static SemaphoreHandle_t    g_tx_done   = NULL;
static volatile bool        g_usb_ready = false;
static bool                 g_usb_opened = false;
static uint8_t              g_rx_scratch[64];

void cdc_usb_callback(usb_event_info_t *p_event_info, usb_hdl_t handle,
                      usb_onoff_t on_off)
{
    (void)handle;
    (void)on_off;

    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    switch (p_event_info->event) {
    case USB_STATUS_CONFIGURED:
        g_usb_ready = true;
        R_USB_Read(&g_basic0_ctrl, g_rx_scratch, sizeof(g_rx_scratch),
                   USB_CLASS_PCDC);
        break;

    case USB_STATUS_READ_COMPLETE:
        if (g_rx_stream != NULL && p_event_info->data_size > 0) {
            xStreamBufferSendFromISR(g_rx_stream, g_rx_scratch,
                                     p_event_info->data_size,
                                     &xHigherPriorityTaskWoken);
        }
        R_USB_Read(&g_basic0_ctrl, g_rx_scratch, sizeof(g_rx_scratch),
                   USB_CLASS_PCDC);
        break;

    case USB_STATUS_WRITE_COMPLETE:
        if (g_tx_done != NULL) {
            xSemaphoreGiveFromISR(g_tx_done, &xHigherPriorityTaskWoken);
        }
        break;

    case USB_STATUS_DETACH:
        g_usb_ready = false;
        break;

    case USB_STATUS_SUSPEND:
        /* Don't kill the connection on suspend — Windows does this during idle */
        break;
```

```
    default:
      break;
    }

    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
}

bool cdc_transport_open(struct uxrCustomTransport *transport)
{
    (void)transport;

    if (g_rx_stream == NULL) {
      g_rx_stream = xStreamBufferCreate(RX_BUFFER_SIZE, RX_TRIGGER_LEVEL);
      if (g_rx_stream == NULL) return false;
    }
    if (g_tx_done == NULL) {
      g_tx_done = xSemaphoreCreateBinary();
      if (g_tx_done == NULL) return false;
    }

    /* Only open USB once — keep it alive across retries */
    if (!g_usb_opened) {
      if (R_USB_Open(&g_basic0_ctrl, &g_basic0_cfg) != FSP_SUCCESS) {
        return false;
      }
      g_usb_opened = true;
    }

    /* Wait up to 5 seconds for host to enumerate us. */
    TickType_t start = xTaskGetTickCount();
    while (!g_usb_ready) {
      if ((xTaskGetTickCount() - start) > pdMS_TO_TICKS(5000)) {
        return false;
      }
      vTaskDelay(pdMS_TO_TICKS(10));
    }
    return true;
}

bool cdc_transport_close(struct uxrCustomTransport *transport)
{
    (void)transport;
    /* Don't close USB — keep it alive so the COM port stays visible */
    return true;
}

size_t cdc_transport_write(struct uxrCustomTransport *transport,
                           const uint8_t *buf, size_t len, uint8_t *err)
```

```
{
  (void)transport;

  if (!g_usb_ready) {
    *err = 1;
    return 0;
  }

  xSemaphoreTake(g_tx_done, 0);

  if (R_USB_Write(&g_basic0_ctrl, (uint8_t *)buf, (uint32_t)len,
            USB_CLASS_PCDC) != FSP_SUCCESS) {
    *err = 1;
    return 0;
  }

  if (xSemaphoreTake(g_tx_done, pdMS_TO_TICKS(WRITE_TIMEOUT_MS)) != pdTRUE) {
    *err = 1;
    return 0;
  }

  return len;
}

size_t cdc_transport_read(struct uxrCustomTransport *transport,
                  uint8_t *buf, size_t len, int timeout, uint8_t *err)
{
  (void)transport;

  if (!g_usb_ready) {
    *err = 1;
    return 0;
  }

  TickType_t ticks = (timeout < 0) ? portMAX_DELAY : pdMS_TO_TICKS(timeout);
  size_t got = xStreamBufferReceive(g_rx_stream, buf, len, ticks);
  if (got == 0) *err = 1;
  return got;
}
```

7.6 microros_stubs.c

POSIX shims for clang libc. clock_gettime/gettimeofday backed by xTaskGetTickCount().

```
#include <time.h>
#include <sys/time.h>
#include <stdint.h>
#include <stdio.h>
#include <unistd.h>

#include "../ra/aws/FreeRTOS/FreeRTOS/Source/include/FreeRTOS.h"
#include "../ra/aws/FreeRTOS/FreeRTOS/Source/include/task.h"
/* tick rate — FreeRTOS default. If you change configTICK_RATE_HZ in FSP,
 * change this to match, or include FreeRTOSConfig.h instead. */
#ifndef configTICK_RATE_HZ
#define configTICK_RATE_HZ 1000
#endif
/* --- Stub for clang libc abort() --- */
void _exit(int status)
{
   (void)status;
   while (1) { }
}
/* --- clock_gettime used by micro-ROS for timestamps --- */
int clock_gettime(clockid_t clk_id, struct timespec *tp)
{
   (void)clk_id;
   uint32_t ticks = xTaskGetTickCount();
   tp->tv_sec  = ticks / configTICK_RATE_HZ;
   tp->tv_nsec = (ticks % configTICK_RATE_HZ) * (1000000000L / configTICK_RATE_HZ);
   return 0;
}
/* --- gettimeofday used by clang libc time() --- */
int gettimeofday(struct timeval *tv, void *tz)
{
   (void)tz;
   if (tv) {
      uint32_t ticks = xTaskGetTickCount();
      tv->tv_sec  = ticks / configTICK_RATE_HZ;
      tv->tv_usec = (ticks % configTICK_RATE_HZ) * (1000000L / configTICK_RATE_HZ);
   }
   return 0;
}
/* --- usleep used by rclc_sleep_ms --- */
int usleep(useconds_t usec)
{
   /* Round up to at least 1 tick so short sleeps still yield. */
   uint32_t ms = (usec + 999) / 1000;
```

```
    if (ms == 0) ms = 1;
    vTaskDelay(pdMS_TO_TICKS(ms));
    return 0;
}
/* --- stderr stub ---
 * rosidl_runtime_c references fprintf(stderr, ...) for error reporting,
 * but clang's libc declares stderr as extern without defining it. Provide
 * a dummy FILE and a no-op fprintf that drops writes. */
static FILE stderr_stub;
//FILE *const stderr = &stderr_stub;
int fprintf(FILE *stream, const char *format, ...)
{
    (void)stream;
    (void)format;
    return 0;
}

/* --- stdout stub ---
 * rcutils_logging_initialize references stdout for console output.
 * Provide a dummy FILE just like stderr above. */
static FILE stdout_stub;
//FILE *const stdout = &stdout_stub;

/* --- isatty stub ---
 * rcutils logging calls isatty() to decide whether to use color codes.
 * Always return 0 (not a terminal). */
int isatty(int fd)
{
    (void)fd;
    return 0;
}
```

7.7 freertos_malloc_override.c

Critical file. Overrides global malloc/free/calloc/realloc so ALL allocations route through FreeRTOS.

```
/**
 * freertos_malloc_override.c
 *
 * Drop this file into your src/ folder. It replaces the standard libc
 * malloc/free/calloc/realloc with versions that route through FreeRTOS
 * pvPortMalloc / vPortFree.
 *
 * WHY: On RA6M5 with clang/ATfE, the default malloc() calls _sbrk(),
 *      which returns -1 because the BSP system heap isn't configured.
 *      You already fixed this for the rcl allocator, but rcutils,
 *      rmw_microxrcedds, and other micro-ROS internals still call
 *      malloc() directly. This file catches ALL of those.
 *
 * REQUIRES: FreeRTOS heap (heap_4.c or heap_5.c) with sufficient
 *           configTOTAL_HEAP_SIZE (you have 200000 — should be fine).
 */

#include <stddef.h>
#include <string.h>

#include "../ra/aws/FreeRTOS/FreeRTOS/Source/include/FreeRTOS.h"

/* ------------------------------------------------------------------ */
/*  Global overrides — the linker will prefer these over libc's weak   */
/*  definitions of malloc/free/calloc/realloc.                         */
/* ------------------------------------------------------------------ */

void *malloc(size_t size)
{
    if (size == 0) {
        return NULL;
    }
    return pvPortMalloc(size);
}

void free(void *ptr)
{
    vPortFree(ptr);  /* vPortFree already handles NULL */
}

void *calloc(size_t nmemb, size_t size)
{
    size_t total = nmemb * size;
```

```
    /* Overflow check */
    if (nmemb != 0 && total / nmemb != size) {
        return NULL;
    }

    void *ptr = pvPortMalloc(total);
    if (ptr != NULL) {
        memset(ptr, 0, total);
    }
    return ptr;
}

void *realloc(void *ptr, size_t new_size)
{
    /*
     * FreeRTOS doesn't have a native realloc. This is a simple
     * allocate-copy-free approach. It works but always copies
     * the full new_size (safe because we can't query the old size
     * from heap_4 without hacking the header). Since micro-ROS
     * rarely reallocs, this is fine.
     *
     * Edge cases handled:
     *   realloc(NULL, size) = malloc(size)
     *   realloc(ptr, 0)    = free(ptr), return NULL
     */
    if (ptr == NULL) {
        return malloc(new_size);
    }

    if (new_size == 0) {
        vPortFree(ptr);
        return NULL;
    }

    void *new_ptr = pvPortMalloc(new_size);
    if (new_ptr != NULL) {
        /*
         * We don't know the old allocation size, so we copy new_size
         * bytes. If the old block was smaller, we're reading past it —
         * but into FreeRTOS heap memory we own, so it's safe (just
         * garbage in the tail). The caller only uses the valid portion.
         *
         * This is the standard workaround for FreeRTOS realloc.
         */
        memcpy(new_ptr, ptr, new_size);
        vPortFree(ptr);
    }
```

```
    return new_ptr;
}
```

7.8 usb_descriptor.c

CDC-ACM descriptors. Device appears as "RA6M5 Sensor Node".

```
/* Standard USB CDC-ACM descriptor for Renesas RA FSP.
 * This is boilerplate from the Renesas USB PCDC example project. */

#include "../ra/fsp/inc/api/r_usb_basic_api.h"
#include "../ra/fsp/inc/instances/r_usb_basic.h"

#define USB_BCDNUM            (0x0200u)
#define USB_RELEASE           (0x0200u)
#define USB_VENDORID          (0x045Bu)   /* Renesas */
#define USB_PRODUCTID         (0x2013u)   /* PCDC example */
#define USB_DCPMAXP           (64u)
#define USB_CONFIGURATION     (0u)
#define USB_DEVICECLASS       (0x02u)     /* Communications Device Class */

/* --- Device Descriptor --- */
uint8_t g_apl_device[] = {
  18u,                          /* bLength */
  USB_DT_DEVICE,                /* bDescriptorType */
  (uint8_t)USB_BCDNUM, (uint8_t)(USB_BCDNUM >> 8), /* bcdUSB */
  USB_DEVICECLASS,              /* bDeviceClass */
  0x00u,                        /* bDeviceSubClass */
  0x00u,                        /* bDeviceProtocol */
  (uint8_t)USB_DCPMAXP,         /* bMaxPacketSize0 */
  (uint8_t)USB_VENDORID, (uint8_t)(USB_VENDORID >> 8),   /* idVendor */
  (uint8_t)USB_PRODUCTID, (uint8_t)(USB_PRODUCTID >> 8), /* idProduct */
  (uint8_t)USB_RELEASE, (uint8_t)(USB_RELEASE >> 8),     /* bcdDevice */
  1,                            /* iManufacturer */
  2,                            /* iProduct */
  3,                            /* iSerialNumber */
  1                             /* bNumConfigurations */
};

/* --- Configuration Descriptor (CDC-ACM, 2 interfaces) --- */
uint8_t g_apl_configuration[] = {
  /* Configuration Descriptor */
  9,                        /* bLength */
  USB_DT_CONFIGURATION,     /* bDescriptorType */
  67, 0,                    /* wTotalLength */
  2,                        /* bNumInterfaces */
  1,                        /* bConfigurationValue */
  0,                        /* iConfiguration */
  0xC0,                     /* bmAttributes: self-powered */
  50,                       /* bMaxPower: 100 mA */
```

```
  /* Interface 0: CDC Control */
  9,                    /* bLength */
  USB_DT_INTERFACE,             /* bDescriptorType */
  0,                    /* bInterfaceNumber */
  0,                    /* bAlternateSetting */
  1,                    /* bNumEndpoints */
  0x02,                   /* bInterfaceClass: CDC */
  0x02,                   /* bInterfaceSubClass: Abstract Control */
  0x01,                   /* bInterfaceProtocol: AT commands */
  0,                    /* iInterface */

  /* CDC Header Functional */
  5, 0x24, 0x00, 0x10, 0x01,

  /* CDC Call Management Functional */
  5, 0x24, 0x01, 0x00, 0x01,

  /* CDC ACM Functional */
  4, 0x24, 0x02, 0x02,

  /* CDC Union Functional */
  5, 0x24, 0x06, 0x00, 0x01,

  /* Endpoint: Interrupt IN (notifications) */
  7, USB_DT_ENDPOINT, 0x83, 0x03, 0x10, 0x00, 0xFF,

  /* Interface 1: CDC Data */
  9,                    /* bLength */
  USB_DT_INTERFACE,             /* bDescriptorType */
  1,                    /* bInterfaceNumber */
  0,                    /* bAlternateSetting */
  2,                    /* bNumEndpoints */
  0x0A,                   /* bInterfaceClass: CDC Data */
  0x00,                   /* bInterfaceSubClass */
  0x00,                   /* bInterfaceProtocol */
  0,                    /* iInterface */

  /* Endpoint: Bulk OUT */
  7, USB_DT_ENDPOINT, 0x01, 0x02, 0x40, 0x00, 0x00,

  /* Endpoint: Bulk IN */
  7, USB_DT_ENDPOINT, 0x82, 0x02, 0x40, 0x00, 0x00
};

/* --- Qualifier & Other-speed Configuration (full-speed only, dummies) --- */
uint8_t g_apl_hs_configuration[] = { 0 };
uint8_t g_apl_qualifier_descriptor[] = { 0 };
```

```
/* --- String Descriptors --- */
/* String 0: LANGID (English US) */
static uint8_t lang_id[] = { 4, USB_DT_STRING, 0x09, 0x04 };

/* String 1: Manufacturer = "Renesas" */
static uint8_t manufacturer[] = {
   16, USB_DT_STRING,
   'R', 0, 'e', 0, 'n', 0, 'e', 0, 's', 0, 'a', 0, 's', 0
};

/* String 2: Product = "RA6M5 Sensor Node" */
static uint8_t product[] = {
   36, USB_DT_STRING,
   'R', 0, 'A', 0, '6', 0, 'M', 0, '5', 0, ' ', 0,
   'S', 0, 'e', 0, 'n', 0, 's', 0, 'o', 0, 'r', 0, ' ', 0,
   'N', 0, 'o', 0, 'd', 0, 'e', 0
};

/* String 3: Serial Number */
static uint8_t serial[] = {
   14, USB_DT_STRING,
   '0', 0, '0', 0, '0', 0, '0', 0, '0', 0, '1', 0
};

uint8_t *g_apl_string_table[] = {
   lang_id,
   manufacturer,
   product,
   serial
};

/* --- The descriptor struct micro-ROS + FSP reference --- */
usb_descriptor_t g_usb_descriptor = {
   .p_device            = g_apl_device,
   .p_config_f          = g_apl_configuration,
   .p_config_h          = g_apl_hs_configuration,
   .p_qualifier         = g_apl_qualifier_descriptor,
   .p_string            = g_apl_string_table,
   .num_string          = 4
};
```

8. Debugging: What Went Wrong and How It Was Fixed

8.1 Bug #1: Broken malloc (rc=10)

Symptom: rclc_support_init() returned RCL_RET_BAD_ALLOC.

Root Cause: clang libc's _sbrk() not wired by FSP. Every malloc() returned NULL.

Fix: freertos_malloc_override.c (Section 7.7) overrides global malloc/free/calloc/realloc.

How this bug was diagnosed: the firmware would reach g_breadcrumb = 3 (rclc_support_init) and never advance. Pausing in the debugger showed that rclc_support_init() returned error code 10, which maps to RCL_RET_BAD_ALLOC. This meant an internal malloc() call returned NULL. The question was why — we had configured the rcl allocator to use pvPortMalloc. The answer is that the rcl-level allocator only affects allocations made through rcl's own allocation API. The underlying libraries (rcutils, rmw_microxrcedds) call the standard C malloc() directly. On bare-metal with the clang toolchain, malloc() calls _sbrk() to grow the heap, but _sbrk() returns (void*)-1 because there is no operating system managing the heap. The fix: override malloc/free/calloc/realloc globally so every allocation in the entire firmware goes through FreeRTOS.

8.2 Bug #2: FreeRTOS Heap = 1024 Bytes

Symptom: Large allocations failed even after fixing malloc.

Root Cause: -DconfigTOTAL_HEAP_SIZE silently ignored by #ifndef guard in generated header.

Fix: Edit ra_cfg/aws/FreeRTOSConfig.h directly. Set to 200000. Verify .bss grew to ~268 KB.

This bug was diagnosed by adding a call to xPortGetFreeHeapSize() right before rclc_support_init() and printing the result via a debug variable. The value was 0 — the heap was completely exhausted before micro-ROS even started. Checking the .map file showed .bss was only about 10 KB, confirming the heap was still at the default 1024 bytes. The -D compiler flag had no effect because of the #ifndef guard. After manually editing FreeRTOSConfig.h to 200000, .bss grew to ~268 KB and xPortGetFreeHeapSize() reported ~180 KB free before initialization, which was enough for micro-ROS.

8.3 Bug #3: rclc_support_init Returns rc=1

Symptom: Error code 1 (RCL_RET_ERROR). USB never enumerated.

Debugging — Breadcrumb Tracing: e² studio debugger has no FreeRTOS task awareness. Always shows idle task when paused. Workaround: volatile int g_breadcrumb set at sequential code points. Pause, read value in Expressions panel.

Error String: rcl_get_error_string() returned garbage ("eF" + nulls). Failure path doesn't set error.

Resolution: Combination of global malloc override + correct heap size. rc=1 was downstream of silent allocation failures.

The breadcrumb tracing technique used to diagnose these bugs deserves additional explanation. The e² studio debugger connects via J-Link to the RA6M5's SWD (Serial Wire Debug) port. However, FreeRTOS

task-awareness is limited in e² studio — when you pause execution, the debugger often lands in the idle task rather than your application task. You cannot reliably set breakpoints inside task code because the scheduler may not be running your task when you pause. The workaround: define a volatile global integer (volatile int g_breadcrumb = 0;) and set it to incrementing values at each step of your initialization sequence. When the firmware hangs, pause the debugger and examine g_breadcrumb in the Variables view. Its value tells you exactly which initialization step succeeded and which one is stuck. Additional diagnostic variables: volatile int g_last_rc stores the most recent error code, volatile int g_rc_check_num stores which RCCHECK macro invocation failed, and volatile char g_last_error_buf[128] stores the error string from rcl_get_error_string().

8.4 Bug #4: USB Device Disappearing

Symptom: Agent connected, 7 topics created, then /dev/ttyACM0 vanished.

Fix: Handle USB_STATUS_SUSPEND as no-op (microros_transports.c line 55–57).

This was the most frustrating bug because it appeared to be random. The agent would connect successfully, all seven topics would appear in ros2 topic list, data would stream for 10-30 seconds, and then /dev/ttyACM0 would vanish from the host. The agent would log "Destroy participant." Reconnecting required unplugging and re-plugging the USB cable, re-running usbipd attach (if using WSL2), and resetting the board.
The root cause: the USB host (Linux) sends periodic SUSPEND signals to USB devices as part of power management. The default USB callback in FSP treats SUSPEND as an error and tears down the USB connection. The fix is to handle USB_STATUS_SUSPEND as a no-op in the USB callback function — simply do nothing and return. The USB host will send a RESUME signal when it wants to communicate again, and the connection continues. This fix is in microros_transports.c. Look for the switch(event) case that handles USB_STATUS_SUSPEND and confirm it contains only a "break" with no cleanup code.

8.5 Bug #5: Permission Denied (errno 13)

Fix: sudo chmod 666 /dev/ttyACM0. Permanent: sudo usermod -a -G dialout $USER.

This is a standard Linux permissions issue. By default, /dev/ttyACM0 is owned by root:dialout and has permissions 660 (rw-rw----). Your user account needs to be in the "dialout" group to access serial devices. The chmod 666 command is a quick fix for the current session; the usermod command is the permanent fix. After running usermod, you must log out and log back in (or reboot) for the group change to take effect. You can verify with "groups" — it should list "dialout" among your groups.

9. Host-Side Setup: WSL2, ROS 2, and the Agent

9.1 Install WSL2 with Ubuntu 22.04

Important: While WSL2 can be used for initial ROS 2 development and testing, this project ultimately required native Linux due to DDS multicast discovery failures across the WSL2 NAT boundary and unreliable USB passthrough via usbipd-win. If you experience intermittent topic discovery issues or random USB disconnections, switch to a native Ubuntu 22.04 installation. See Section 12.4 for the full explanation.

Admin PowerShell:

```
wsl --install -d Ubuntu-22.04
```

Reboot. Enter later: wsl -d Ubuntu-22.04

After running wsl --install -d Ubuntu-22.04, Windows will download the Ubuntu image and set up WSL2. You will be prompted to create a Linux username and password. This is your Linux user account inside WSL2, separate from your Windows account. After the initial setup, you can enter WSL2 at any time from PowerShell with "wsl -d Ubuntu-22.04" or by opening the "Ubuntu 22.04" app from the Start menu.
Verify WSL2 is using version 2 (not version 1) by running "wsl -l -v" in PowerShell. The output should show your distribution with VERSION 2. WSL1 uses a translation layer and has different networking behavior; WSL2 runs a real Linux kernel in a VM and is required for ROS 2.

9.2 Install ROS 2 Humble

```
sudo apt update && sudo apt install locales -y
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8


sudo apt install software-properties-common -y
sudo add-apt-repository universe
sudo apt update && sudo apt install curl -y
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
  -o /usr/share/keyrings/ros-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) \
  signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] \
  http://packages.ros.org/ros2/ubuntu jammy main" \
  | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null


sudo apt update
sudo apt install ros-humble-ros-base python3-rosdep build-essential -y
echo 'source /opt/ros/humble/setup.bash' >> ~/.bashrc
source ~/.bashrc
```

Verify the installation: run "ros2 topic list" in a new terminal. It should return an empty list (no error). Run "ros2 --version" to confirm the installed version. If "ros2" is not found, make sure you sourced the setup file: "source /opt/ros/humble/setup.bash" and that the echo command added it to your ~/.bashrc. Note: ros-humble-ros-base is the minimal ROS 2 installation (no GUI tools). For debugging with rqt_plot or rviz2, install the full desktop version instead: "sudo apt install ros-humble-desktop". This is optional and uses significant disk space (~2 GB).

Where to find it: ROS 2 Humble install guide: https://docs.ros.org/en/humble/Installation/Ubuntu-Install-Debs.html

9.3 Build the Micro-ROS Agent

```
mkdir -p ~/microros_ws/src && cd ~/microros_ws/src
git clone -b humble https://github.com/micro-ROS/micro_ros_setup.git
cd ~/microros_ws
source /opt/ros/humble/setup.bash
colcon build
source install/local_setup.bash
ros2 run micro_ros_setup create_agent_ws.sh
ros2 run micro_ros_setup build_agent.sh
```

The agent build takes approximately 5-10 minutes. The script clones the micro-ROS Agent source, resolves its dependencies using vcstool, and builds everything with colcon. If the build fails with memory errors, try closing other applications — the build is memory-intensive. If it fails with network errors, the script is trying to clone Git repositories and your network connection may be unreliable.
After building, verify the agent works: "ros2 run micro_ros_agent micro_ros_agent --help". This should print the agent's command-line options. You do not need the board connected yet — you are just verifying the agent binary was built correctly.

```
echo 'source ~/microros_ws/install/local_setup.bash' >> ~/.bashrc
```

Where to find it: micro-ROS Agent build instructions: https://github.com/micro-ROS/micro-ROS-Agent

9.4 USB Passthrough (usbipd-win)

Where to find it: usbipd-win releases: https://github.com/dorssel/usbipd-win/releases — Microsoft WSL USB documentation: https://learn.microsoft.com/en-us/windows/wsl/connect-usb

Install from https://github.com/dorssel/usbipd-win/releases. Admin PowerShell:

```
usbipd bind --hardware-id 045b:2013
usbipd attach --wsl --hardware-id 045b:2013 --auto-attach --unplugged
```

Leave running. --auto-attach --unplugged intercepts before Windows grabs the device.

The hardware ID 045b:2013 is the USB VID:PID for Renesas devices. Find your board's specific ID by running "usbipd list" in PowerShell while the board is connected. Look for an entry that says "Renesas" or matches the board name. If your board has a different VID:PID, substitute it in the commands above. The --auto-attach --unplugged flags tell usbipd to automatically forward the USB device to WSL2 whenever it appears, even if it is unplugged and re-plugged. This saves you from manually re-binding after every board reset. The usbipd PowerShell window must stay open for auto-attach to work. If you close it, you will need to re-run the attach command.

Inside WSL2, verify the device appeared: "ls -la /dev/ttyACM*". You should see /dev/ttyACM0. If nothing appears, check that the usbipd bind and attach commands succeeded (no error messages), and that the board is running firmware with USB-CDC enabled (Phase A or Phase B).

9.5 Run the Agent and Verify

Terminal 1:

```
sudo chmod 666 /dev/ttyACM0
ros2 run micro_ros_agent micro_ros_agent serial --dev /dev/ttyACM0
```

Press board reset. Agent shows 1 participant, 7 topics, 7 publishers, 7 datawriters.

Terminal 2:

```
ros2 topic list
ros2 topic echo /sensors/room_temp
```

WSL2 DDS Limitation

ros2 topic list may only show /parameter_events and /rosout. This is a WSL2 networking issue, NOT a firmware bug. Use agent -v6 to verify. Works on native Linux.

10. Deployment: Podman Container

Where to find it: Podman (rootless container engine): https://podman.io — Podman installation guide: https://podman.io/docs/installation

10.1 Containerfile

```
mkdir -p ~/microros_container
cat > ~/microros_container/Containerfile << 'EOF'
FROM ros:humble-ros-base
RUN apt-get update && apt-get install -y python3-pip \
  && pip3 install vcstool && rm -rf /var/lib/apt/lists/*
SHELL ["/bin/bash", "-c"]
RUN mkdir -p /microros_ws/src \
  && cd /microros_ws/src \
  && source /opt/ros/humble/setup.bash \
  && git clone -b humble https://github.com/micro-ROS/micro_ros_setup.git \
  && cd /microros_ws && colcon build \
  && source install/local_setup.bash \
  && ros2 run micro_ros_setup create_agent_ws.sh \
  && ros2 run micro_ros_setup build_agent.sh
ENTRYPOINT ["/bin/bash", "-c", \
 "source /opt/ros/humble/setup.bash && \
  source /microros_ws/install/local_setup.bash && \
  ros2 run micro_ros_agent micro_ros_agent serial --dev /dev/ttyACM0"]
EOF
```

10.2 Build and Run

```
cd ~/microros_container && podman build -t microros-agent .
podman run --rm --device=/dev/ttyACM0 microros-agent
```

The --device=/dev/ttyACM0 flag passes the USB serial device into the container so the agent can access it. If the device is at a different path (e.g., /dev/ttyACM1), substitute accordingly. The --rm flag removes the container when it exits, keeping your system clean.

Why Podman instead of Docker? Podman is rootless by default — it does not require a daemon running as root, which is a security advantage. The Containerfile syntax is identical to Dockerfile syntax, so you can substitute "docker" for "podman" in all commands if you prefer Docker. The container image is approximately 2-3 GB after building because it includes the full ROS 2 installation and the compiled agent.

To run the containerized agent in the background: "podman run -d --device=/dev/ttyACM0 --name microros microros-agent". To view its logs: "podman logs -f microros". To stop it: "podman stop microros". This setup is useful for deploying the agent on a dedicated Linux machine (e.g., a Raspberry Pi) where you want it to start automatically and run unattended.

For automatic startup on boot, create a systemd user service that runs the podman container. Create ~/.config/systemd/user/microros-agent.service with [Service] ExecStart=podman run --rm --device=/dev/ttyACM0 microros-agent, then enable it with "systemctl --user enable microros-agent".

11. Troubleshooting Quick Reference

No COM port appears when connecting J12: The firmware is not running, or the USB stack was not initialized. Check that the firmware was flashed and is running (not paused in debugger). In Phase A, ensure R_USB_Open() is called and R_USB_EventGet() is being polled in the main loop. In Phase B, ensure the USB PCDC stack was added in FSP and the project was regenerated.

Agent starts but shows 0 participants: The board is not sending XRCE-DDS frames. Check: (1) Is the correct serial port specified? (2) Is the firmware actually running the micro-ROS initialization? Use breadcrumb values to check. (3) Is the USB cable connected to J12 (not J11)? (4) Did you chmod the device?

Agent shows "Create participant" then immediately "Destroy participant": The firmware is crashing during or after initialization. Common cause: insufficient heap or stack. Check g_breadcrumb — if it reaches 999, read g_last_rc and g_last_error_buf for details.

ros2 topic list shows no topics but agent shows active participants: DDS multicast discovery failure. This is almost always a WSL2 networking issue. Switch to native Linux (Section 12.4). If already on native Linux, check that no firewall is blocking UDP multicast (ports 7400-7500).

/dev/ttyACM0 disappears after 10-30 seconds: USB SUSPEND not handled. Check that your USB callback treats USB_STATUS_SUSPEND as a no-op (Bug #4, Section 8.4).

Build error "conflicting CPU architectures 17/2": libmicroros.a was built for the wrong ARM architecture. Rebuild with -mcpu=cortex-m33. Architecture 17 is ARMv8-M (correct for RA6M5), architecture 2 is ARMv7-M (wrong). See Section 6.3.

Linker error "undefined reference to clock_gettime" (or gettimeofday, usleep, etc.): Missing POSIX stubs. Add microros_stubs.c (Section 7.6) to your project. Each undefined function needs a stub implementation.

rclc_support_init returns rc=10 (RCL_RET_BAD_ALLOC): malloc() is returning NULL. Ensure freertos_malloc_override.c is in the project (Section 7.7) AND that configTOTAL_HEAP_SIZE is set to 200000 in FreeRTOSConfig.h (not via -D flag).

Sensor readings are all zero: I2C communication failure. Check wiring: SDA to P414, SCL to P415. Check that FSP I2C is configured for Channel 2. Try reading each sensor individually to isolate which one is not responding. Verify sensor power (3.3V on VCC, GND connected).

Build succeeds but firmware immediately hard-faults: Stack overflow in the FreeRTOS task. Increase stack size (currently 40000). Also check that you are not calling micro-ROS functions from outside the FreeRTOS task context (e.g., from hal_entry or an ISR).

Symptom	Likely Cause	Fix
Board not in Device Manager	Power-only cable or firmware hung	Different cable; debugger check R_USB_Open
"rcl/rcl.h not found"	Include path missing	Add include/ in Compiler → Includes
"conflicting CPU architectures"	libmicroros.a wrong ARM target	Rebuild with -mcpu=cortex-m33
RCL_RET_BAD_ALLOC (rc=10)	malloc returning NULL	Add freertos_malloc_override.c
Heap resets to 1024	FSP regenerated FreeRTOSConfig.h	Re-edit after every Generate Project Content
Agent: errno 13	Permission denied	sudo chmod 666 /dev/ttyACM0

ros2 topic list empty	WSL2 DDS discovery broken	Not a bug; agent -v6; works on native Linux
USB disappears	SUSPEND treated as DETACH	Handle USB_STATUS_SUSPEND as no-op

12. Lessons Learned

12.1 USB Polling Is Non-Negotiable

In bare-metal mode, R_USB_EventGet() must be called in a tight loop or the USB state machine never advances. Not prominently documented by Renesas.

In bare-metal mode, the Renesas USB stack does not use interrupts for state machine management. Instead, R_USB_EventGet() must be called in a polling loop to process USB events (enumeration, configuration, data transfer completion, suspend/resume). If this function is not called frequently enough (at least every few milliseconds), the USB enumeration handshake will time out and the host will report "Device not recognized." In the Phase A code, the main while(1) loop runs continuously with only a 1ms delay, ensuring USB is serviced frequently. In Phase B (FreeRTOS), the USB interrupt handler takes over event processing, so this is less of a concern — but understanding it helps when debugging USB issues.

12.2 Fix the Allocator at Every Level

Fixing only the rcl-level allocator is insufficient. rcutils and rmw_microxrcedds call malloc() directly. The global override in freertos_malloc_override.c is the complete fix.

The micro-ROS memory allocation architecture has three levels: (1) the application-level allocator you configure in rclc_support_init (this uses pvPortMalloc), (2) the rcutils allocator used internally by rcl and rclc (also configurable, but must be set separately), and (3) direct malloc() calls inside rmw_microxrcedds and the XRCE-DDS client (not configurable — they call the C standard library directly). The global override in freertos_malloc_override.c catches level 3 by replacing the C library's weak malloc/free symbols with strong versions that call pvPortMalloc/vPortFree. This is the only reliable way to ensure every allocation in the entire firmware uses the FreeRTOS heap.

12.3 Compiler -D Flags Can Be Silently Ignored

#ifndef guards in generated headers mean -D flags have no effect if the header defines the value first. Read the actual files.

12.4 WSL2 Is Not Linux

USB requires usbipd-win. DDS multicast discovery is broken. Assume any USB or topic discovery issue is WSL2. Test on native Linux.

The root cause is architectural. WSL2 runs a real Linux kernel, but inside a lightweight Hyper-V virtual machine with its own virtual network adapter (vEthernet). This means WSL2 sits behind a NAT — it has a different IP address from the Windows host. DDS (the middleware ROS 2 uses for node discovery) relies on UDP multicast to find other nodes on the network. UDP multicast does not reliably cross the WSL2 NAT boundary.

The micro-ROS agent inside WSL2 could receive XRCE-DDS messages from the board over the serial connection, but its DDS announcements often failed to reach ros2 topic list running in the same WSL2 instance. The symptoms were intermittent — sometimes ros2 topic list showed all seven topics, sometimes it showed none, sometimes a few. This made it nearly impossible to distinguish infrastructure problems from firmware bugs. Hours were spent debugging "firmware issues" that were actually WSL2 networking issues.

USB passthrough added a second layer of fragility. usbipd-win shares USB devices from Windows to WSL2 through a chain: a Windows service captures the USB device, an IP-based protocol forwards it, and a virtual USB host controller inside the WSL2 kernel presents it. This chain was unreliable — the board would randomly disconnect from /dev/ttyACM0 mid-session, requiring re-binding with usbipd attach --wsl each time. Sometimes the device would not re-enumerate at all without physically unplugging and re-plugging the USB cable.

The solution was a laptop running native Ubuntu 22.04 — not a VM, not WSL2, but a real Linux installation on bare metal. The difference was immediate: the USB device appeared as /dev/ttyACM0 the moment it was plugged in, the micro-ROS agent connected on the first try, DDS discovery worked perfectly, and ros2 topic list showed all seven topics within seconds. What had taken hours of debugging under WSL2 worked on the first attempt on native Linux.

The lesson: for any project involving USB devices and DDS-based middleware (which includes all of ROS 2 and micro-ROS), use native Linux. WSL2 is excellent for many development tasks, but the combination of NAT-broken multicast and fragile USB passthrough makes it unsuitable for micro-ROS development. Start on native Linux from day one.

12.5 Budget a Full Day for Library Generation

Keep a clean PATH, exclude C++ packages, verify the ARM architecture tag.

The library generation process is the most time-consuming and error-prone step. Plan for multiple attempts. Each attempt takes 40-90 minutes, and the error messages are often unhelpful ("recipe for target X failed" with 500 lines of compiler output). The strategy that works: (1) Start with a completely clean environment (a fresh WSL2 instance or Docker container). (2) Install only the required tools (cmake, colcon, python3, the ARM cross-compiler). (3) Strip the PATH to remove all Windows directories. (4) Set the correct ARM flags explicitly. (5) Exclude C++-only packages. (6) Run the build and walk away for 90 minutes. If it fails, read the error output carefully — it is usually one of: wrong PATH (Windows directories leaking in), wrong ARM flags (architecture mismatch), or a missing dependency (python3 package or cmake version). After a successful build, save the libmicroros.a and include/ directory somewhere safe so you never have to rebuild unless you change the target architecture or need a different micro-ROS version.

13. Project File Reference

13.1 Bare-Metal Project

```
Sensor_Project_USB_Version/
 ├─ src/
 |  ├─ hal_entry.c           ← ALL application logic
 |  ├─ hal_warmstart.c       ← Auto-generated startup
 |  └─ r_usb_pcdc_descriptor.c ← USB descriptors
 ├─ ra_gen/                  ← FSP-generated drivers
 ├─ ra_cfg/                  ← FSP config headers
 └─ configuration.xml        ← FSP Configurator data
```

13.2 Micro-ROS Project

```
Sensor_Project_ROS/
 ├─ src/
 |  ├─ new_thread0_entry.c   ← Micro-ROS node (7 publishers)
 |  ├─ sensors.c / sensors.h  ← Sensor abstraction
 |  ├─ microros_transports.c / .h ← USB-CDC transport
 |  ├─ microros_stubs.c      ← POSIX shims
 |  ├─ freertos_malloc_override.c ← Global allocator override
 |  └─ usb_descriptor.c      ← CDC-ACM descriptors
 ├─ micro_ros_renesas2estudio_component/
 |  └─ libmicroros/
 |     ├─ libmicroros.a      ← Pre-compiled library
 |     └─ include/           ← Micro-ROS headers
 ├─ ra_cfg/aws/FreeRTOSConfig.h  ← MUST have heap = 200000
 └─ configuration.xml        ← FSP Configurator data
```

14. Glossary

Term	Definition
e² studio	Renesas' Eclipse-based IDE for RA microcontrollers
FSP	Flexible Software Package — Renesas' HAL and driver package
FreeRTOS	Real-time OS providing threads, semaphores, memory management
micro-ROS	ROS 2 for microcontrollers with limited resources
libmicroros.a	Pre-compiled static library: entire micro-ROS client stack
micro-ROS agent	Host-side serial-to-DDS bridge
XRCE-DDS	eXtremely Resource Constrained Environments DDS — lightweight protocol between client and agent
USB-CDC (PCDC)	USB Communications Device Class — virtual serial port
DDS	Data Distribution Service — ROS 2's middleware protocol for topic discovery and transport
WSL2	Windows Subsystem for Linux 2 — VM running Linux inside Windows
usbipd-win	Forwards USB devices from Windows into WSL2
Podman	Container runtime (Docker alternative)
QWIIC	SparkFun's 4-pin JST I2C connector system
Breadcrumb Tracing	Debug technique: volatile int set at code points, read in debugger to track execution
pvPortMalloc/vPortFree	FreeRTOS heap allocation functions
_sbrk()	POSIX function for heap extension; not implemented by FSP for clang
CDR	Common Data Representation — serialization format used by DDS for message encoding